🌯 Informe Técnico Maestro: Código Fuente Íntegro y Explicación de AlgeKebab

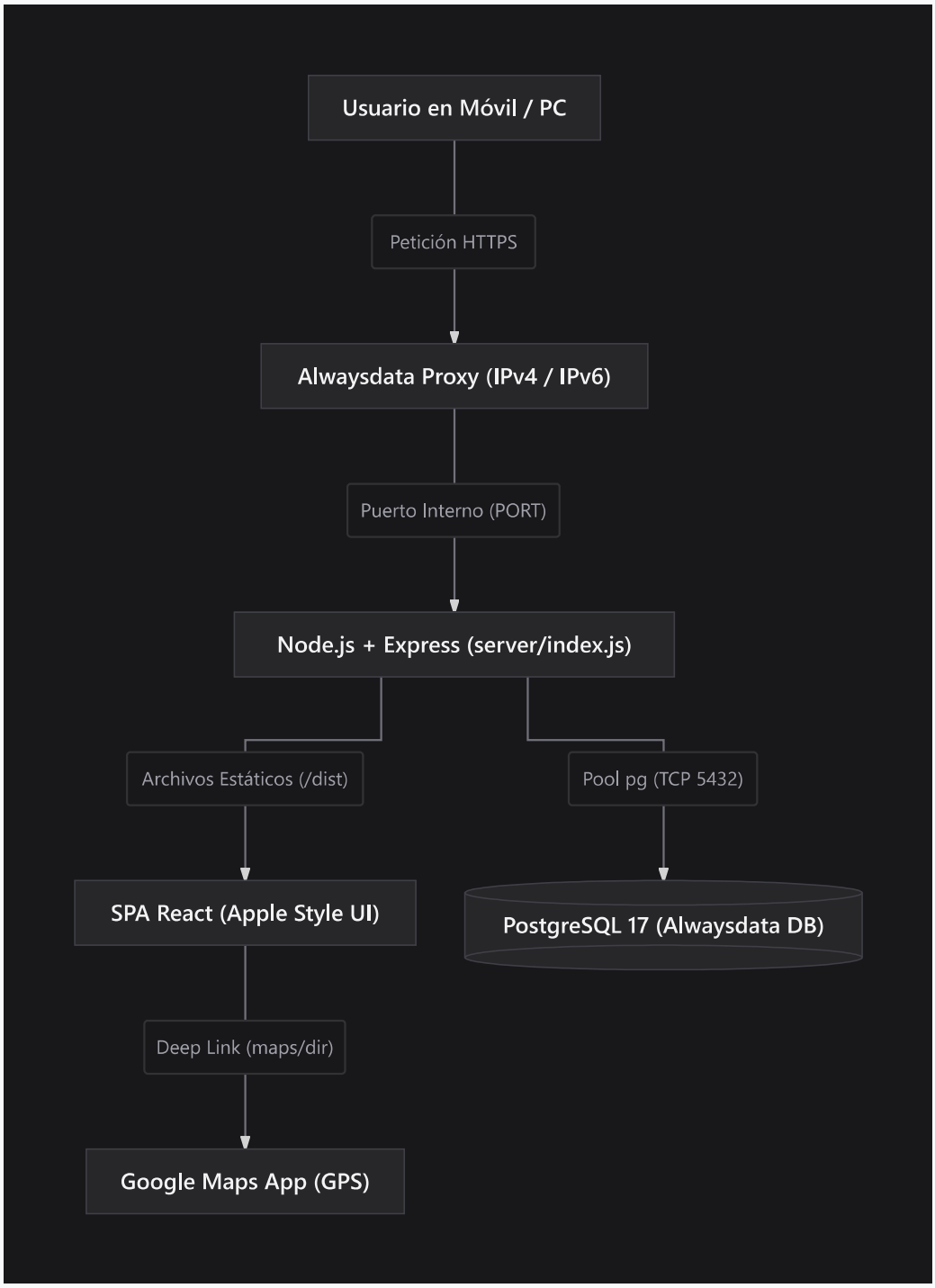
Este documento contiene el **100% del código fuente** de la aplicación **AlgeKebab**, organizado archivo por archivo con su explicación técnica detallada, propósito, flujos de datos y lógica interna.

📑 Índice de Contenidos

1. Arquitectura General
2. Base de Datos: `database.sql`
3. Backend: Servidor Express
    - 3.1. `server/index.js`
    - 3.2. `server/db.js`
    - 3.3. `server/formula.js`
    - 3.4. `server/keepAlive.js`
    - 3.5. `server/routes/auth.js`
    - 3.6. `server/routes/kebabs.js`
    - 3.7. `server/routes/ratings.js`
4. Frontend: Estética Minimalista Apple / Samsung (Rojo y Blanco)
    - 4.1. Configuración: `client/tailwind.config.js` e `index.css`
    - 4.2. Estado Global: `client/src/context/AuthContext.jsx`
    - 4.3. Lógica de Ponderación: `client/src/utils/formula.js`
    - 4.4. Página Principal: `client/src/App.jsx`
    - 4.5. Logotipo: `client/src/components/Logo.jsx`
    - 4.6. Barra Superior: `client/src/components/Header.jsx`
    - 4.7. Tarjeta de Kebab: `client/src/components/KebabCard.jsx`
    - 4.8. Ficha Detallada: `client/src/components/KebabDetailModal.jsx`
    - 4.9. Formulario de Votación: `client/src/components/RatingModal.jsx`
    - 4.10. Modal de Autenticación: `client/src/components/AuthModal.jsx`
    - 4.11. Modal de Perfil: `client/src/components/ProfileModal.jsx`

    - 4.12. Modal Añadir Kebab: `client/src/components/AddKebabModal.jsx`
    - 4.13. Modal de Criterios: `client/src/components/FormulaInfoModal.jsx`
    - 4.14. Punto de Entrada React: `client/src/main.jsx`
    - 4.15. Plantilla HTML Principal: `client/index.html`
    - 4.16. Configuración de Vite: `client/vite.config.js`
5. Configuración de Paquetes y Dependencias
    - 5.1. `package.json` (Raíz)
    - 5.2. `client/package.json` (Frontend)
6. Variables de Entorno y Conexión en Alwaysdata

1. Arquitectura General



2. Base de Datos: `database.sql`

Archivo que define la estructura relacional de tablas en PostgreSQL y precarga los 8 establecimientos de Algeciras.

```sql
-- ============================================================
-- ESQUEMA DE BASE DE DATOS: RUTA DEL KEBAB ALGECIRAS
-- ============================================================

-- 1. Tabla de Usuarios
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    display_name VARCHAR(100),
    avatar_color VARCHAR(20) DEFAULT '#f59e0b',
    is_hidden BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Tabla de Kebabs (Establecimientos en Algeciras)
CREATE TABLE IF NOT EXISTS kebabs (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(255) NOT NULL,
    zone VARCHAR(100) NOT NULL,
    google_maps_url TEXT,
    description TEXT,
    image_url TEXT,
    phone VARCHAR(30),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 3. Tabla de Puntuaciones y Reseñas
CREATE TABLE IF NOT EXISTS ratings (
    id SERIAL PRIMARY KEY,
    kebab_id INTEGER REFERENCES kebabs(id) ON DELETE CASCADE,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    kebab_score NUMERIC(3, 1) NOT NULL CHECK (kebab_score >= 0 AND kebab_score <= 10),
    ubicacion_score NUMERIC(3, 1) NOT NULL CHECK (ubicacion_score >= 0 AND ubicacion_score
<= 10),
    atencion_score NUMERIC(3, 1) NOT NULL CHECK (atencion_score >= 0 AND atencion_score <=
10),
    calidad_precio_score NUMERIC(3, 1) NOT NULL CHECK (calidad_precio_score >= 0 AND
calidad_precio_score <= 10),
    local_presentacion_score NUMERIC(3, 1) NOT NULL CHECK (local_presentacion_score >= 0
AND local_presentacion_score <= 10),
    final_score NUMERIC(3, 1) NOT NULL CHECK (final_score >= 0 AND final_score <= 10),
    comment TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_user_kebab UNIQUE (kebab_id, user_id)
);

-- 4. Tabla de Keep-Alive
CREATE TABLE IF NOT EXISTS _keep_alive (
    id INTEGER PRIMARY KEY DEFAULT 1,
    movie VARCHAR(255) NOT NULL,
    last_ping TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    ping_count INTEGER DEFAULT 1,
    note TEXT
);

-- Datos iniciales: Los 8 Kebabs de Algeciras
INSERT INTO kebabs (id, name, address, zone, google_maps_url, description, image_url)
VALUES
(1, 'Kebab La Isla (Comida Rápida)', 'Calle Sindicalista Luis Cobos', 'Zona Puerto / Isla
Verde', 'https://maps.google.com/?q=La+Isla+Comida+Rapida+Algeciras', 'Mítico punto de
reunión con dürüms bien cargados, shawarma y opciones clásicas.',
'https://images.unsplash.com/photo-1529006557810-274b9b2fc783?
auto=format&fit=crop&w=800&q=80'),
(2, 'Kebab Dchiqui', 'Av. Europa, 2 (Edif. Las Palomas)', 'Las Palomas',
'https://maps.google.com/?q=Kebab+Dchiqui+Algeciras', 'Considerado por muchos de los
mejores de Algeciras y Campo de Gibraltar. Salsas caseras insuperables.',
'https://images.unsplash.com/photo-1561651823-34feb02250e4?
auto=format&fit=crop&w=800&q=80'),
(3, 'Turkish Kebab Algeciras', 'Av. Virgen del Carmen, 33', 'Zona Puerto / Centro',
'https://maps.google.com/?q=Turkish+Kebab+Algeciras', 'Gran cantidad de carne sabrosa y
salsa picante auténtica muy potente.', 'https://images.unsplash.com/photo-1626777552726-
4a6b54c97e46?auto=format&fit=crop&w=800&q=80'),
(4, 'King Döner Kebab y Pizza', 'Av. Virgen del Carmen, 29', 'Zona Puerto',
'https://maps.google.com/?q=King+Doner+Kebab+Algeciras', 'Destaca por sus patatas fritas
naturales e ingredientes frescos que marcan la diferencia.',
'https://images.unsplash.com/photo-1594212699903-ec8a3eca50f5?
auto=format&fit=crop&w=800&q=80'),
(5, 'Mi Casa Döner Kebab', 'Paseo Marítimo de Getares', 'Getares / San García',
'https://maps.google.com/?q=Mi+Casa+Doner+Kebab+Getares+Algeciras', 'Perfecto para cenar
frente a la playa o en la zona sur. Dürüms y bandejas con queso.',
'https://images.unsplash.com/photo-1555396273-367ea4eb4db5?
auto=format&fit=crop&w=800&q=80'),
(6, 'Original Kebab', 'C.C. Puerta Europa (Planta Alta)', 'Puerta Europa',
'https://maps.google.com/?q=Original+Kebab+Puerta+Europa+Algeciras', 'Ideal tras tarde de
compras o cine. Menús completos de dürum y platos turcos.',
'https://images.unsplash.com/photo-1541592106381-b31e9677c0e5?
auto=format&fit=crop&w=800&q=80'),
```

```
(7, 'El Palacio Comida Rápida', 'Calle Sindicalista Luis Cobos, 5', 'Zona Puerto /
Bajadilla', 'https://maps.google.com/?q=El+Palacio+Comida+Rapida+Algeciras', 'Shawarmas
sabrosos, camperos y especialidades con trato muy cercano y limpio.',
'https://images.unsplash.com/photo-1544025162-d76694265947?
auto=format&fit=crop&w=800&q=80'),
(8, 'Tasty''s Algeciras', 'Av. Europa / San García', 'San García / Europa',
'https://maps.google.com/?q=Tasty+Algeciras', 'Fusión de tacos franceses con salsa de queso
y döner kebab con toque moderno.', 'https://images.unsplash.com/photo-1565299585323-
38d6b0865b47?auto=format&fit=crop&w=800&q=80')
ON CONFLICT (id) DO NOTHING;

SELECT setval(pg_get_serial_sequence('kebabs', 'id'), coalesce(max(id), 1)) FROM kebabs;
```

> **Explicación**: El esquema crea integridad referencial (`ON DELETE CASCADE` borra puntuaciones asociadas si se borra un kebab), y la restricción `unique_user_kebab` previene votos duplicados del mismo usuario.

3. Backend: Servidor Express

3.1. `server/index.js`

Es el punto de entrada de la aplicación. Configura middlewares, rutas y sirve la aplicación compilada.

```
javascript
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const { initDb, supabase, pool, dbMode, readLocalDb, writeLocalDb } = require('./db');
const { startKeepAlive } = require('./keepAlive');

const { router: authRouter } = require('./routes/auth');
const kebabsRouter = require('./routes/kebabs');
const ratingsRouter = require('./routes/ratings');

const app = express();
const PORT = process.env.PORT || 3001;

// Middlewares
app.use(cors());
app.use(express.json());

// API Routes
app.use('/api/auth', authRouter);
app.use('/api/kebabs', kebabsRouter);
app.use('/api/ratings', ratingsRouter);

// Endpoint Keep-Alive / Health check
app.get('/api/health', (req, res) => {
  res.json({ status: 'ok', service: 'AlgeKebab API', timestamp: new Date().toISOString()
});
});

app.get('/api/ping', (req, res) => {
  res.send('pong');
});

// Servir frontend compilado en producción
const clientDistPath = path.join(__dirname, '../client/dist');
app.use(express.static(clientDistPath));

app.get('*', (req, res) => {
  const indexHtml = path.join(clientDistPath, 'index.html');
  if (require('fs').existsSync(indexHtml)) {
    res.sendFile(indexHtml);
  } else {
    res.json({
      message: 'AlgeKebab API funcionando correctamente.',
      frontend: 'El frontend aún no está compilado.'
```

```
    });
  }
});

// Arrancar servidor e inicializar DB + Bot Keep-Alive
const HOST = process.env.IP || (process.env.NODE_ENV === 'production' ? '::' : '0.0.0.0');

initDb().then(() => {
  app.listen(PORT, HOST, () => {
    console.log(`================================================`);
    console.log(`🥙 Servidor AlgeKebab escuchando en ${HOST}:${PORT}`);
    console.log(`🚀 Listo para Alwaysdata, Render o desarrollo local`);
    console.log(`================================================`);

    // Iniciar bot de películas invisible
    startKeepAlive({ supabase, pool, dbMode, readLocalDb, writeLocalDb });
  });
}).catch(err => {
  console.error('Error al iniciar el servidor:', err);
});
```

> **Explicación**: El servidor define `HOST` dinámicamente: si Alwaysdata le pasa `process.env.IP` (como `fd00::...` en IPv6), se vincula a esa IP; en desarrollo local se vincula a `0.0.0.0`. Además, `app.get('*')` asegura que cualquier ruta del navegador sirva `index.html` (imprescindible en React SPA).

3.2. `server/db.js`

Gestiona la conexión con PostgreSQL mediante `Pool` de `pg`.

```javascript
const fs = require('fs');
const path = require('path');
const { createClient } = require('@supabase/supabase-js');
const { Pool } = require('pg');
const bcrypt = require('bcryptjs');

const SUPABASE_URL = process.env.SUPABASE_URL || '';
const SUPABASE_KEY = process.env.SUPABASE_KEY || '';
const DATABASE_URL = process.env.DATABASE_URL || '';

let supabase = null;
let pool = null;

let dbMode = 'local'; // 'supabase' | 'postgres' | 'local'

if (SUPABASE_URL && SUPABASE_KEY) {
  try {
    supabase = createClient(SUPABASE_URL, SUPABASE_KEY);
    dbMode = 'supabase';
    console.log('⚡ Conectando a Supabase mediante API Client:', SUPABASE_URL);
  } catch (e) {
    console.error('Error configurando Supabase client:', e.message);
  }
} else if (DATABASE_URL) {
  try {
    pool = new Pool({
      connectionString: DATABASE_URL,
      ssl: DATABASE_URL.includes('localhost') ? false : { rejectUnauthorized: false }
    });
    dbMode = 'postgres';
    console.log('🐘 Conectando a PostgreSQL mediante conexión directa');
  } catch (e) {
    console.error('Error configurando PostgreSQL pool:', e.message);
  }
} else {
  console.log('📁 Modo base de datos: Almacenamiento local (JSON).');
}

const localDbPath = path.join(__dirname, '../data/local_db.json');

// Lista base de los 8 kebabs
const INITIAL_KEBABS = [
  { id: 1, name: 'Kebab La Isla (Comida Rápida)', address: 'Calle Sindicalista Luis Cobos',
zone: 'Zona Puerto / Isla Verde', google_maps_url: 'https://maps.google.com/?
q=La+Isla+Comida+Rapida+Algeciras', description: 'Mítico punto de reunión con dürüms bien
```

```
cargados, shawarma y opciones clásicas.', image_url: 'https://images.unsplash.com/photo-1529006557810-274b9b2fc783?auto=format&fit=crop&w=800&q=80', created_at: new Date().toISOString() },
  { id: 2, name: 'Kebab Dchiqui', address: 'Av. Europa, 2 (Edif. Las Palomas)', zone: 'Las Palomas', google_maps_url: 'https://maps.google.com/?q=Kebab+Dchiqui+Algeciras', description: 'Considerado por muchos de los mejores de Algeciras y Campo de Gibraltar. Salsas caseras insuperables.', image_url: 'https://images.unsplash.com/photo-1561651823-34feb02250e4?auto=format&fit=crop&w=800&q=80', created_at: new Date().toISOString() },
  { id: 3, name: 'Turkish Kebab Algeciras', address: 'Av. Virgen del Carmen, 33', zone: 'Zona Puerto / Centro', google_maps_url: 'https://maps.google.com/?q=Turkish+Kebab+Algeciras', description: 'Gran cantidad de carne sabrosa y salsa picante auténtica muy potente.', image_url: 'https://images.unsplash.com/photo-1626777552726-4a6b54c97e46?auto=format&fit=crop&w=800&q=80', created_at: new Date().toISOString() },
  { id: 4, name: 'King Döner Kebab y Pizza', address: 'Av. Virgen del Carmen, 29', zone: 'Zona Puerto', google_maps_url: 'https://maps.google.com/?q=King+Doner+Kebab+Algeciras', description: 'Destaca por sus patatas fritas naturales e ingredientes frescos que marcan la diferencia.', image_url: 'https://images.unsplash.com/photo-1594212699903-ec8a3eca50f5?auto=format&fit=crop&w=800&q=80', created_at: new Date().toISOString() },
  { id: 5, name: 'Mi Casa Döner Kebab', address: 'Paseo Marítimo de Getares', zone: 'Getares / San García', google_maps_url: 'https://maps.google.com/?q=Mi+Casa+Doner+Kebab+Getares+Algeciras', description: 'Perfecto para cenar frente a la playa o en la zona sur. Dürüms y bandejas con queso.', image_url: 'https://images.unsplash.com/photo-1555396273-367ea4eb4db5?auto=format&fit=crop&w=800&q=80', created_at: new Date().toISOString() },
  { id: 6, name: 'Original Kebab', address: 'C.C. Puerta Europa (Planta Alta)', zone: 'Puerta Europa', google_maps_url: 'https://maps.google.com/?q=Original+Kebab+Puerta+Europa+Algeciras', description: 'Ideal tras tarde de compras o cine. Menús completos de dürum y platos turcos.', image_url: 'https://images.unsplash.com/photo-1541592106381-b31e9677c0e5?auto=format&fit=crop&w=800&q=80', created_at: new Date().toISOString() },
  { id: 7, name: 'El Palacio Comida Rápida', address: 'Calle Sindicalista Luis Cobos, 5', zone: 'Zona Puerto / Bajadilla', google_maps_url: 'https://maps.google.com/?q=El+Palacio+Comida+Rapida+Algeciras', description: 'Shawarmas sabrosos, camperos y especialidades con trato muy cercano y limpio.', image_url: 'https://images.unsplash.com/photo-1544025162-d76694265947?auto=format&fit=crop&w=800&q=80', created_at: new Date().toISOString() },
  { id: 8, name: 'Tasty\'s Algeciras', address: 'Av. Europa / San García', zone: 'San García / Europa', google_maps_url: 'https://maps.google.com/?q=Tasty+Algeciras', description: 'Fusión de tacos franceses con salsa de queso y döner kebab con toque moderno.', image_url: 'https://images.unsplash.com/photo-1565299585323-38d6b0865b47?auto=format&fit=crop&w=800&q=80', created_at: new Date().toISOString() }
];

function readLocalDb() {
  if (!fs.existsSync(localDbPath)) return { users: [], kebabs: INITIAL_KEBABS, ratings: [] };
```

```
  try {
    return JSON.parse(fs.readFileSync(localDbPath, 'utf8'));
  } catch {
    return { users: [], kebabs: INITIAL_KEBABS, ratings: [] };
  }
}

function writeLocalDb(data) {
  fs.writeFileSync(localDbPath, JSON.stringify(data, null, 2), 'utf8');
}

async function initDb() {
  if (dbMode === 'postgres') {
    console.log('Iniciando esquema en PostgreSQL...');
    const sqlPath = path.join(__dirname, '../database.sql');
    if (fs.existsSync(sqlPath)) {
      const sql = fs.readFileSync(sqlPath, 'utf8');
      await pool.query(sql);
      console.log('Tablas PostgreSQL creadas/actualizadas.');
    }
  }
}

// Métodos de Usuario
async function findUserByUsername(username) {
  if (dbMode === 'postgres') {
    const res = await pool.query('SELECT * FROM users WHERE LOWER(username) = LOWER($1)',
[username]);
    return res.rows[0] || null;
  }
  const db = readLocalDb();
  return db.users.find(u => u.username.toLowerCase() === username.toLowerCase()) || null;
}

async function findUserById(id) {
  if (dbMode === 'postgres') {
    const res = await pool.query('SELECT id, username, display_name, avatar_color,
created_at FROM users WHERE id = $1', [id]);
    return res.rows[0] || null;
  }
  const db = readLocalDb();
  const u = db.users.find(user => user.id === id);
  if (!u) return null;
  const { password_hash, ...safe } = u;
  return safe;
}
```

```
async function createUser({ username, display_name, password_hash, avatar_color }) {
  if (dbMode === 'postgres') {
    const res = await pool.query(
      `INSERT INTO users (username, display_name, password_hash, avatar_color)
       VALUES ($1, $2, $3, $4)
       RETURNING id, username, display_name, avatar_color, created_at`,
      [username, display_name || username, password_hash, avatar_color || '#dc2626']
    );
    return res.rows[0];
  }
  const db = readLocalDb();
  const newUser = {
    id: db.users.length > 0 ? Math.max(...db.users.map(u => u.id)) + 1 : 1,
    username,
    display_name: display_name || username,
    password_hash,
    avatar_color: avatar_color || '#dc2626',
    created_at: new Date().toISOString()
  };
  db.users.push(newUser);
  writeLocalDb(db);
  const { password_hash: _, ...safe } = newUser;
  return safe;
}

async function updateUser(id, { username, password_hash }) {
  if (dbMode === 'postgres') {
    let query = 'UPDATE users SET ';
    const params = [];
    let idx = 1;
    if (username) {
      query += `username = $${idx}, display_name = $${idx}, `;
      params.push(username);
      idx++;
    }
    if (password_hash) {
      query += `password_hash = $${idx}, `;
      params.push(password_hash);
      idx++;
    }
    query = query.slice(0, -2);
    query += ` WHERE id = $${idx} RETURNING id, username, display_name, avatar_color,
created_at`;
    params.push(id);
    const res = await pool.query(query, params);
    return res.rows[0];
```

```
  }
  const db = readLocalDb();
  const user = db.users.find(u => u.id === id);
  if (!user) return null;
  if (username) { user.username = username; user.display_name = username; }
  if (password_hash) user.password_hash = password_hash;
  writeLocalDb(db);
  const { password_hash: _, ...safe } = user;
  return safe;
}

// Obtener todos los kebabs con ratings consolidados
async function getAllKebabsWithRatings() {
  if (dbMode === 'postgres') {
    const kebabsRes = await pool.query('SELECT * FROM kebabs ORDER BY id ASC');
    const ratingsRes = await pool.query(`
      SELECT r.*, u.username, u.display_name, u.avatar_color
      FROM ratings r
      JOIN users u ON r.user_id = u.id
      ORDER BY r.created_at DESC
    `);
    return kebabsRes.rows.map(kebab => {
      const kebabRatings = ratingsRes.rows.filter(r => r.kebab_id === kebab.id);
      const count = kebabRatings.length;
      const avg = count > 0 ? kebabRatings.reduce((acc, r) => acc +
parseFloat(r.final_score), 0) / count : null;
      return {
        ...kebab,
        average_score: avg !== null ? Math.round(avg * 10) / 10 : null,
        ratings_count: count,
        ratings: kebabRatings
      };
    });
  }

  const db = readLocalDb();
  return db.kebabs.map(kebab => {
    const kebabRatings = db.ratings
      .filter(r => r.kebab_id === kebab.id)
      .map(r => {
        const user = db.users.find(u => u.id === r.user_id) || { username: 'Anónimo',
avatar_color: '#9ca3af' };
        return {
          ...r,
          username: user.username,
          display_name: user.display_name || user.username,
```

```
          avatar_color: user.avatar_color
        };
      });
    const count = kebabRatings.length;
    const avg = count > 0 ? kebabRatings.reduce((acc, r) => acc +
parseFloat(r.final_score), 0) / count : null;
    return {
      ...kebab,
      average_score: avg !== null ? Math.round(avg * 10) / 10 : null,
      ratings_count: count,
      ratings: kebabRatings
    };
  });
}

async function addKebab({ name, address, zone, google_maps_url, description, image_url }) {
  if (dbMode === 'postgres') {
    const res = await pool.query(
      `INSERT INTO kebabs (name, address, zone, google_maps_url, description, image_url)
       VALUES ($1, $2, $3, $4, $5, $6)
       RETURNING *`,
      [name, address, zone, google_maps_url, description, image_url]
    );
    return res.rows[0];
  }
  const db = readLocalDb();
  const newKebab = {
    id: db.kebabs.length > 0 ? Math.max(...db.kebabs.map(k => k.id)) + 1 : 1,
    name, address, zone, google_maps_url, description, image_url,
    created_at: new Date().toISOString()
  };
  db.kebabs.push(newKebab);
  writeLocalDb(db);
  return newKebab;
}

async function saveRating({ kebab_id, user_id, kebab_score, ubicacion_score,
atencion_score, calidad_precio_score, local_presentacion_score, final_score, comment }) {
  if (dbMode === 'postgres') {
    const res = await pool.query(
      `INSERT INTO ratings (kebab_id, user_id, kebab_score, ubicacion_score,
atencion_score, calidad_precio_score, local_presentacion_score, final_score, comment,
updated_at)
       VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, NOW())
       ON CONFLICT (kebab_id, user_id)
       DO UPDATE SET
```

```
          kebab_score = EXCLUDED.kebab_score,
          ubicacion_score = EXCLUDED.ubicacion_score,
          atencion_score = EXCLUDED.atencion_score,
          calidad_precio_score = EXCLUDED.calidad_precio_score,
          local_presentacion_score = EXCLUDED.local_presentacion_score,
          final_score = EXCLUDED.final_score,
          comment = EXCLUDED.comment,
          updated_at = NOW()
        RETURNING *`,
       [kebab_id, user_id, kebab_score, ubicacion_score, atencion_score,
calidad_precio_score, local_presentacion_score, final_score, comment]
    );
    return res.rows[0];
  }
  const db = readLocalDb();
  const existingIdx = db.ratings.findIndex(r => r.kebab_id === kebab_id && r.user_id ===
user_id);
  const now = new Date().toISOString();
  let ratingRecord;
  if (existingIdx >= 0) {
    ratingRecord = {
      ...db.ratings[existingIdx],
      kebab_score, ubicacion_score, atencion_score, calidad_precio_score,
local_presentacion_score, final_score, comment,
      updated_at: now
    };
    db.ratings[existingIdx] = ratingRecord;
  } else {
    ratingRecord = {
      id: db.ratings.length > 0 ? Math.max(...db.ratings.map(r => r.id)) + 1 : 1,
      kebab_id, user_id, kebab_score, ubicacion_score, atencion_score,
calidad_precio_score, local_presentacion_score, final_score, comment,
      created_at: now, updated_at: now
    };
    db.ratings.push(ratingRecord);
  }
  writeLocalDb(db);
  return ratingRecord;
}

async function deleteRating(kebab_id, user_id) {
  if (dbMode === 'postgres') {
    await pool.query('DELETE FROM ratings WHERE kebab_id = $1 AND user_id = $2', [kebab_id,
user_id]);
    return true;
  }
```

```
  const db = readLocalDb();
  db.ratings = db.ratings.filter(r => !(r.kebab_id === kebab_id && r.user_id === user_id));
  writeLocalDb(db);
  return true;
}

module.exports = {
  dbMode,
  pool,
  supabase,
  initDb,
  readLocalDb,
  writeLocalDb,
  findUserByUsername,
  findUserById,
  createUser,
  updateUser,
  getAllKebabsWithRatings,
  addKebab,
  saveRating,
  deleteRating
};
```

3.3. `server/formula.js`

Lógica de cálculo ponderado en el backend.

```
javascript
const CRITERIA_CONFIG = {
  kebab: { maxPoints: 6.0, multiplier: 0.6 },
  ubicacion: { maxPoints: 1.0, multiplier: 0.1 },
  atencion: { maxPoints: 1.0, multiplier: 0.1 },
  calidad_precio: { maxPoints: 1.0, multiplier: 0.1 },
  local_presentacion: { maxPoints: 1.0, multiplier: 0.1 }
};

function calculateFinalScore(scores) {
  const kebabRaw = parseFloat(scores.kebab_score) || 0;
  const ubicacionRaw = parseFloat(scores.ubicacion_score) || 0;
  const atencionRaw = parseFloat(scores.atencion_score) || 0;
  const calidadPrecioRaw = parseFloat(scores.calidad_precio_score) || 0;
  const localPresentacionRaw = parseFloat(scores.local_presentacion_score) || 0;

  const contribKebab = (kebabRaw / 10) * CRITERIA_CONFIG.kebab.maxPoints;
  const contribUbicacion = (ubicacionRaw / 10) * CRITERIA_CONFIG.ubicacion.maxPoints;
  const contribAtencion = (atencionRaw / 10) * CRITERIA_CONFIG.atencion.maxPoints;
  const contribCalidadPrecio = (calidadPrecioRaw / 10) *
CRITERIA_CONFIG.calidad_precio.maxPoints;
  const contribLocalPresentacion = (localPresentacionRaw / 10) *
CRITERIA_CONFIG.local_presentacion.maxPoints;

  const total = contribKebab + contribUbicacion + contribAtencion + contribCalidadPrecio +
contribLocalPresentacion;

  return Math.round(total * 10) / 10;
}

module.exports = {
  CRITERIA_CONFIG,
  calculateFinalScore
};
```

3.4. `server/keepAlive.js`

Trabajador en segundo plano que escribe un ping cada 5 minutos en PostgreSQL para evitar suspensión de recursos en proveedores en la nube.

```javascript
const MOVIES = [
  'El Padrino', 'Pulp Fiction', 'El Gran Lebowski', 'Matrix', 'Interstellar',
  'Gladiator', 'Uno de los nuestros', 'El Club de la Lucha', 'Blade Runner 2049',
  'Scarface', 'El Bueno, el Feo y el Malo', 'Origen (Inception)', 'Casino',
  'Reservoir Dogs', 'El Caballero Oscuro', 'Seven', 'Terminator 2', 'Parásitos',
  'Apocalypse Now', 'Memento'
];

let pingCount = 0;

function getRandomMovie() {
  const idx = Math.floor(Math.random() * MOVIES.length);
  return MOVIES[idx];
}

function startKeepAlive({ supabase, pool, dbMode, readLocalDb, writeLocalDb }) {
  const INTERVAL_MS = 5 * 60 * 1000;

  console.log('🤖 Bot Keep-Alive (CineBot) iniciado: enviará actividad a Supabase cada 5 minutos.');

  const timer = setInterval(async () => {
    pingCount++;
    const movie = getRandomMovie();
    const timestamp = new Date().toISOString();

    try {
      if (dbMode === 'supabase' && supabase) {
        await supabase.from('_keep_alive').upsert([
          { id: 1, movie, last_ping: timestamp, ping_count: pingCount, note: 'Ping automático' }
        ], { onConflict: 'id' });
      } else if (dbMode === 'postgres' && pool) {
        await pool.query(`
          INSERT INTO _keep_alive (id, movie, last_ping, ping_count)
          VALUES (1, $1, NOW(), $2)
          ON CONFLICT (id) DO UPDATE SET movie = $1, last_ping = NOW(), ping_count = $2
        `, [movie, pingCount]);
      } else {
        const db = readLocalDb();
        db._keep_alive = { id: 1, movie, last_ping: timestamp, ping_count: pingCount };
        writeLocalDb(db);
      }
    } catch (err) {
      console.warn(`[🤖 CineBot] Error enviando ping:`, err.message);
```

```
    }
  }, INTERVAL_MS);

  return timer;
}

module.exports = { startKeepAlive };
```

3.5. `server/routes/auth.js`

Rutas de autenticación: registro, login, consulta de sesión y modificación de perfil.

```javascript
const express = require('express');
const router = express.Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const {
  findUserByUsername,
  findUserById,
  createUser,
  updateUser
} = require('../db');

const JWT_SECRET = process.env.JWT_SECRET || 'algekebab-super-secret-key-2026';

function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Acceso no autorizado. Inicia sesión.' });
  }

  jwt.verify(token, JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ error: 'Sesión expirada o token inválido.' });
    }
    req.user = user;
    next();
  });
}

// 1. Registro
router.post('/register', async (req, res) => {
  try {
    const { username, password } = req.body;
    if (!username || !password) {
      return res.status(400).json({ error: 'El nombre de usuario y la contraseña son
obligatorios.' });
    }
    if (username.trim().length < 2) {
      return res.status(400).json({ error: 'El nombre de usuario debe tener al menos 2
caracteres.' });
    }
    if (password.length < 4) {
      return res.status(400).json({ error: 'La contraseña debe tener al menos 4
caracteres.' });
```

```
    }

    const cleanUsername = username.trim();
    const existing = await findUserByUsername(cleanUsername);
    if (existing) {
      return res.status(400).json({ error: 'Ese nombre de usuario ya está cogido. Elige
otro.' });
    }

    const colors = ['#dc2626', '#b91c1c', '#ef4444', '#991b1b'];
    const randomColor = colors[Math.floor(Math.random() * colors.length)];
    const passwordHash = await bcrypt.hash(password, 10);

    const newUser = await createUser({
      username: cleanUsername,
      display_name: cleanUsername,
      password_hash: passwordHash,
      avatar_color: randomColor
    });

    const token = jwt.sign({ id: newUser.id, username: newUser.username }, JWT_SECRET, {
expiresIn: '365d' });
    return res.status(201).json({ user: newUser, token });
  } catch (err) {
    console.error('Error in /register:', err);
    res.status(500).json({ error: 'Error al registrar el usuario.' });
  }
});

// 2. Login
router.post('/login', async (req, res) => {
  try {
    const { username, password } = req.body;
    if (!username || !password) {
      return res.status(400).json({ error: 'Introduce usuario y contraseña.' });
    }

    const cleanUsername = username.trim();
    const user = await findUserByUsername(cleanUsername);
    if (!user) {
      return res.status(401).json({ error: 'Usuario no encontrado.' });
    }

    const valid = await bcrypt.compare(password, user.password_hash);
    if (!valid) {
      return res.status(401).json({ error: 'Contraseña incorrecta.' });
    }
```

```
    const token = jwt.sign({ id: user.id, username: user.username }, JWT_SECRET, {
expiresIn: '365d' });
    const { password_hash, ...safeUser } = user;

    return res.json({ user: safeUser, token });
  } catch (err) {
    console.error('Error in /login:', err);
    res.status(500).json({ error: 'Error al iniciar sesión.' });
  }
});

// 3. Usuario actual
router.get('/me', authenticateToken, async (req, res) => {
  try {
    const user = await findUserById(req.user.id);
    if (!user) return res.status(404).json({ error: 'Usuario no encontrado.' });
    return res.json({ user });
  } catch (err) {
    res.status(500).json({ error: 'Error al consultar la sesión.' });
  }
});

// 4. Actualizar perfil
router.put('/profile', authenticateToken, async (req, res) => {
  try {
    const { new_username, new_password, current_password } = req.body;
    const userId = req.user.id;

    const user = await findUserById(userId);
    if (!user) return res.status(404).json({ error: 'Usuario no encontrado' });

    if (current_password) {
      const fullUser = await findUserByUsername(user.username);
      const valid = await bcrypt.compare(current_password, fullUser.password_hash);
      if (!valid) return res.status(400).json({ error: 'La contraseña actual no es
correcta.' });
    }

    let updatedUsername = user.username;
    let updatedHash = undefined;

    if (new_username && new_username.trim() !== user.username) {
      const cleanNew = new_username.trim();
      const check = await findUserByUsername(cleanNew);
      if (check && check.id !== userId) {
        return res.status(400).json({ error: 'Ese nombre de usuario ya está en uso.' });
      }
```

```
      updatedUsername = cleanNew;
    }

    if (new_password) {
      if (new_password.length < 4) return res.status(400).json({ error: 'La nueva
contraseña debe tener al menos 4 caracteres.' });
      updatedHash = await bcrypt.hash(new_password, 10);
    }

    const updated = await updateUser(userId, {
      username: updatedUsername,
      password_hash: updatedHash
    });

    const newToken = jwt.sign({ id: userId, username: updatedUsername }, JWT_SECRET, {
expiresIn: '365d' });
    return res.json({ user: updated, token: newToken, message: 'Perfil actualizado
correctamente.' });
  } catch (err) {
    console.error('Error updating profile:', err);
    res.status(500).json({ error: 'Error al actualizar el perfil.' });
  }
});

module.exports = { router, authenticateToken };
```

3.6. `server/routes/kebabs.js`

Rutas para listar kebabs o registrar uno nuevo.

```javascript
const express = require('express');
const router = express.Router();
const { getAllKebabsWithRatings, addKebab } = require('../db');
const { authenticateToken } = require('./auth');

// 1. Obtener todos los kebabs con sus puntuaciones y detalles
router.get('/', async (req, res) => {
  try {
    const kebabs = await getAllKebabsWithRatings();
    return res.json(kebabs);
  } catch (err) {
    console.error('Error fetching kebabs:', err);
    res.status(500).json({ error: 'Error al obtener la lista de kebabs.' });
  }
});

// 2. Obtener un kebab concreto por ID
router.get('/:id', async (req, res) => {
  try {
    const kebabId = parseInt(req.params.id);
    const kebabs = await getAllKebabsWithRatings();
    const kebab = kebabs.find(k => k.id === kebabId);
    if (!kebab) return res.status(404).json({ error: 'Kebab no encontrado' });
    return res.json(kebab);
  } catch (err) {
    res.status(500).json({ error: 'Error al consultar kebab.' });
  }
});

// 3. Añadir nuevo kebab
router.post('/', authenticateToken, async (req, res) => {
  try {
    const { name, address, zone, google_maps_url, description, image_url } = req.body;

    if (!name || !zone) {
      return res.status(400).json({ error: 'El nombre y la zona en Algeciras son
obligatorios.' });
    }

    const defaultImage = 'https://images.unsplash.com/photo-1529006557810-274b9b2fc783?
auto=format&fit=crop&w=800&q=80';

    const newKebab = await addKebab({
      name,
      address: address || 'Algeciras',
```

```
      zone,
      google_maps_url: google_maps_url || '',
      description: description || '',
      image_url: image_url || defaultImage
    });

    return res.status(201).json(newKebab);
  } catch (err) {
    console.error('Error creating kebab:', err);
    res.status(500).json({ error: 'Error al registrar el kebab.' });
  }
});

module.exports = router;
```

3.7. `server/routes/ratings.js`

Rutas de votación individual de los amigos.

```javascript
const express = require('express');
const router = express.Router();
const { saveRating, deleteRating } = require('../db');
const { authenticateToken } = require('./auth');
const { calculateFinalScore } = require('../formula');

// Crear o actualizar la puntuación del usuario actual para un kebab
router.post('/', authenticateToken, async (req, res) => {
  try {
    const userId = req.user.id;
    const {
      kebab_id,
      kebab_score,
      ubicacion_score,
      atencion_score,
      calidad_precio_score,
      local_presentacion_score,
      comment
    } = req.body;

    if (!kebab_id) {
      return res.status(400).json({ error: 'Debes indicar el kebab que estás puntuando.'
});
    }

    const scores = {
      kebab_score: Math.min(10, Math.max(0, parseFloat(kebab_score) || 0)),
      ubicacion_score: Math.min(10, Math.max(0, parseFloat(ubicacion_score) || 0)),
      atencion_score: Math.min(10, Math.max(0, parseFloat(atencion_score) || 0)),
      calidad_precio_score: Math.min(10, Math.max(0, parseFloat(calidad_precio_score) ||
0)),
      local_presentacion_score: Math.min(10, Math.max(0,
parseFloat(local_presentacion_score) || 0))
    };

    const final_score = calculateFinalScore(scores);

    const saved = await saveRating({
      kebab_id: parseInt(kebab_id),
      user_id: userId,
      ...scores,
      final_score,
      comment: comment || ''
    });
```

```
    return res.json({ rating: saved, message: '¡Puntuación guardada con éxito!' });
  } catch (err) {
    console.error('Error saving rating:', err);
    res.status(500).json({ error: 'Error al registrar la puntuación.' });
  }
});

// Eliminar la puntuación de un usuario para un kebab
router.delete('/:kebabId', authenticateToken, async (req, res) => {
  try {
    const userId = req.user.id;
    const kebabId = parseInt(req.params.kebabId);

    await deleteRating(kebabId, userId);
    return res.json({ message: 'Puntuación eliminada correctamente.' });
  } catch (err) {
    console.error('Error deleting rating:', err);
    res.status(500).json({ error: 'Error al eliminar la puntuación.' });
  }
});

module.exports = router;
```

4. Frontend Minimalista: Apple / Samsung (Rojo y Blanco)

4.1. Configuración Tailwind y CSS

`client/tailwind.config.js`

```
javascript
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {
      colors: {
        primary: {
          50: '#fef2f2',
          100: '#fee2e2',
          200: '#fecaca',
          300: '#fca5a5',
          400: '#f87171',
          500: '#ef4444',
          600: '#dc2626',
          700: '#b91c1c',
          800: '#991b1b',
          900: '#7f1d1d',
          950: '#450a0a',
        },
      },
      fontFamily: {
        sans: ['SF Pro Display', 'Inter', 'system-ui', '-apple-system', 'sans-serif'],
      },
    },
  },
  plugins: [],
}
```

`client/src/index.css`

```
css
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  font-family: 'SF Pro Display', 'Inter', system-ui, -apple-system, sans-serif;
  color-scheme: light;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

/* Subtle scrollbar */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}
::-webkit-scrollbar-track {
  background: #f5f5f7;
}
::-webkit-scrollbar-thumb {
  background: #d1d1d6;
  border-radius: 9999px;
}
::-webkit-scrollbar-thumb:hover {
  background: #aeaeb2;
}

.scrollbar-none::-webkit-scrollbar {
  display: none;
}
.scrollbar-none {
  -ms-overflow-style: none;
  scrollbar-width: none;
}

@keyframes fadeIn {
  from { opacity: 0; transform: scale(0.99); }
  to { opacity: 1; transform: scale(1); }
}

.animate-fade-in {
  animation: fadeIn 0.15s ease-out forwards;
}
```

```
/* Range input slider styling */
input[type="range"] {
  -webkit-appearance: none;
  appearance: none;
  background: #e5e5e5;
  border-radius: 9999px;
  height: 6px;
  outline: none;
}
input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 20px;
  height: 20px;
  border-radius: 50%;
  background: #dc2626;
  cursor: pointer;
  border: 2px solid #ffffff;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.15);
  transition: transform 0.1s ease;
}
input[type="range"]::-webkit-slider-thumb:hover {
  transform: scale(1.1);
}
```

4.2. Estado Global: `client/src/context/AuthContext.jsx`

Gestiona la autenticación con persistencia local (`localStorage`) y validación de tokens.

```javascript
import React, { createContext, useContext, useState, useEffect } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    try {
      const saved = localStorage.getItem('algekebab_user');
      return saved ? JSON.parse(saved) : null;
    } catch {
      return null;
    }
  });

  const [token, setToken] = useState(() => {
    return localStorage.getItem('algekebab_token') || null;
  });

  const [loading, setLoading] = useState(true);

  // Verificar validez del token en el servidor al cargar
  useEffect(() => {
    async function verifySession() {
      if (!token) {
        setLoading(false);
        return;
      }
      try {
        const res = await fetch('/api/auth/me', {
          headers: { Authorization: `Bearer ${token}` }
        });
        if (res.ok) {
          const data = await res.json();
          setUser(data.user);
          localStorage.setItem('algekebab_user', JSON.stringify(data.user));
        } else {
          logout();
        }
      } catch (err) {
        console.warn('No se pudo verificar la sesión en el servidor:', err);
      } finally {
        setLoading(false);
      }
    }
```

```
    verifySession();
  }, [token]);

  const login = async (username, password) => {
    const res = await fetch('/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });

    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Error al iniciar sesión');

    setUser(data.user);
    setToken(data.token);
    localStorage.setItem('algekebab_token', data.token);
    localStorage.setItem('algekebab_user', JSON.stringify(data.user));
    return data.user;
  };

  const register = async (username, password) => {
    const res = await fetch('/api/auth/register', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });

    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Error en el registro');

    setUser(data.user);
    setToken(data.token);
    localStorage.setItem('algekebab_token', data.token);
    localStorage.setItem('algekebab_user', JSON.stringify(data.user));
    return data.user;
  };

  const logout = () => {
    setUser(null);
    setToken(null);
    localStorage.removeItem('algekebab_token');
    localStorage.removeItem('algekebab_user');
  };

  const updateProfile = async (updateData) => {
    if (!token) throw new Error('No hay sesión activa');
```

```
    const res = await fetch('/api/auth/profile', {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${token}`
      },
      body: JSON.stringify(updateData)
    });

    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Error al actualizar perfil');

    setUser(data.user);
    if (data.token) {
      setToken(data.token);
      localStorage.setItem('algekebab_token', data.token);
    }
    localStorage.setItem('algekebab_user', JSON.stringify(data.user));
    return data;
  };

  return (
    <AuthContext.Provider value={{ user, token, loading, login, register, logout,
updateProfile }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth debe usarse dentro de un AuthProvider');
  return context;
}
```

4.3. Lógica de Ponderación: `client/src/utils/formula.js`

```
javascript
export const CRITERIA_CONFIG = [
  {
    id: 'kebab_score',
    label: 'Kebab (Carne, Pan y Sabor)',
    icon: '🌯',
    maxPoints: 6.0,
    multiplier: 0.6,
    weightLabel: 'Hasta 6 ptos',
    percentage: '60%',
    description: 'Calidad de la carne, pan crujiente, salsas y sabor general'
  },
  {
    id: 'ubicacion_score',
    label: 'Ubicación',
    icon: '📍',
    maxPoints: 1.0,
    multiplier: 0.1,
    weightLabel: 'Hasta 1 pto',
    percentage: '10%',
    description: 'Zona de Algeciras, cercanía y accesibilidad'
  },
  {
    id: 'atencion_score',
    label: 'Atención del moro',
    icon: '🤝',
    maxPoints: 1.0,
    multiplier: 0.1,
    weightLabel: 'Hasta 1 pto',
    percentage: '10%',
    description: 'Trato al cliente, rapidez, simpatía y generosidad'
  },
  {
    id: 'calidad_precio_score',
    label: 'Calidad-Precio',
    icon: '💶',
    maxPoints: 1.0,
    multiplier: 0.1,
    weightLabel: 'Hasta 1 pto',
    percentage: '10%',
    description: 'Precio respecto al tamaño, cantidad y sabor'
  },
  {
```

```
    id: 'local_presentacion_score',
    label: 'Local y Presentación',
    icon: '🏪',
    maxPoints: 1.0,
    multiplier: 0.1,
    weightLabel: 'Hasta 1 pto',
    percentage: '10%',
    description: 'Limpieza del local, envoltura del durum y ambiente'
  }
];

export function calculateFormulaScore(scores) {
  const k = parseFloat(scores.kebab_score) || 0;
  const u = parseFloat(scores.ubicacion_score) || 0;
  const a = parseFloat(scores.atencion_score) || 0;
  const p = parseFloat(scores.calidad_precio_score) || 0;
  const l = parseFloat(scores.local_presentacion_score) || 0;

  const total = (k * 0.6) + (u * 0.1) + (a * 0.1) + (p * 0.1) + (l * 0.1);
  return Math.round(total * 10) / 10;
}

export function calculateContribution(rawScore, maxPoints) {
  const val = parseFloat(rawScore) || 0;
  return Math.round((val / 10) * maxPoints * 100) / 100;
}

export function getScoreBadgeColor(score) {
  if (score === null || score === undefined) return 'bg-gray-100 text-gray-500 border-gray-
200';
  if (score >= 8.5) return 'bg-red-600 text-white border-red-600';
  if (score >= 7.0) return 'bg-red-500 text-white border-red-500';
  if (score >= 5.0) return 'bg-orange-500 text-white border-orange-500';
  return 'bg-gray-400 text-white border-gray-400';
}
```

4.4. Página Principal: `client/src/App.jsx`

Renderiza la cabecera, la sección Hero minimalista estilo Apple, filtros, chips de zonas, listado de kebabs y el pie de página limpio.

```jsx
import React, { useState, useEffect, useMemo } from 'react';
import Header from './components/Header';
import KebabCard from './components/KebabCard';
import KebabDetailModal from './components/KebabDetailModal';
import RatingModal from './components/RatingModal';
import AuthModal from './components/AuthModal';
import ProfileModal from './components/ProfileModal';
import AddKebabModal from './components/AddKebabModal';
import FormulaInfoModal from './components/FormulaInfoModal';
import { useAuth } from './context/AuthContext';
import { Search, Filter, Star } from 'lucide-react';

export default function App() {
  const { user, token } = useAuth();

  const [kebabs, setKebabs] = useState([]);
  const [loading, setLoading] = useState(true);
  const [searchTerm, setSearchTerm] = useState('');
  const [selectedZone, setSelectedZone] = useState('Todos');
  const [sortBy, setSortBy] = useState('rating');

  const [selectedKebab, setSelectedKebab] = useState(null);
  const [ratingKebab, setRatingKebab] = useState(null);
  const [showAuthModal, setShowAuthModal] = useState(false);
  const [showProfileModal, setShowProfileModal] = useState(false);
  const [showAddModal, setShowAddModal] = useState(false);
  const [showFormulaModal, setShowFormulaModal] = useState(false);

  const fetchKebabs = async () => {
    try {
      setLoading(true);
      const res = await fetch('/api/kebabs');
      if (res.ok) {
        const data = await res.json();
        setKebabs(data);

        if (selectedKebab) {
          const updatedCurrent = data.find(k => k.id === selectedKebab.id);
          if (updatedCurrent) setSelectedKebab(updatedCurrent);
        }
      }
    } catch (err) {
      console.error('Error al cargar kebabs:', err);
    } finally {
      setLoading(false);
```

```
  }
};

useEffect(() => {
  fetchKebabs();
}, []);

const handleSaveRating = async (ratingPayload) => {
  if (!token) {
    setShowAuthModal(true);
    return;
  }

  const res = await fetch('/api/ratings', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`
    },
    body: JSON.stringify(ratingPayload)
  });

  const data = await res.json();
  if (!res.ok) throw new Error(data.error || 'Error al guardar puntuación');
  await fetchKebabs();
};

const handleDeleteRating = async (kebabId) => {
  if (!token) return;
  try {
    const res = await fetch(`/api/ratings/${kebabId}`, {
      method: 'DELETE',
      headers: { Authorization: `Bearer ${token}` }
    });
    if (res.ok) await fetchKebabs();
  } catch (err) {
    console.error('Error deleting rating:', err);
  }
};

const handleAddKebab = async (newKebabData) => {
  if (!token) {
    setShowAuthModal(true);
    return;
  }

  const res = await fetch('/api/kebabs', {
```

```
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${token}`
      },
      body: JSON.stringify(newKebabData)
    });

    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Error al añadir kebab');
    await fetchKebabs();
  };

  const zones = useMemo(() => {
    const list = new Set(kebabs.map(k => k.zone).filter(Boolean));
    return ['Todos', ...Array.from(list)];
  }, [kebabs]);

  const filteredKebabs = useMemo(() => {
    return kebabs
      .filter(k => {
        const matchesSearch =
          k.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
          k.zone.toLowerCase().includes(searchTerm.toLowerCase()) ||
          (k.description &&
k.description.toLowerCase().includes(searchTerm.toLowerCase()));

        const matchesZone = selectedZone === 'Todos' || k.zone === selectedZone;
        return matchesSearch && matchesZone;
      })
      .sort((a, b) => {
        if (sortBy === 'rating') {
          const scoreA = a.average_score !== null ? a.average_score : -1;
          const scoreB = b.average_score !== null ? b.average_score : -1;
          return scoreB - scoreA;
        }
        if (sortBy === 'votes') {
          return (b.ratings?.length || 0) - (a.ratings?.length || 0);
        }
        if (sortBy === 'name') {
          return a.name.localeCompare(b.name);
        }
        return 0;
      });
  }, [kebabs, searchTerm, selectedZone, sortBy]);

  const stats = useMemo(() => {
```

```
    const totalVotes = kebabs.reduce((acc, k) => acc + (k.ratings?.length || 0), 0);
    const topKebab = [...kebabs]
      .filter(k => k.average_score !== null)
      .sort((a, b) => b.average_score - a.average_score)[0];

    const myVotesCount = user
      ? kebabs.reduce((acc, k) => acc + (k.ratings?.some(r => r.user_id === user.id) ? 1 :
0), 0)
      : 0;

    return { totalVotes, topKebab, myVotesCount };
  }, [kebabs, user]);

  return (
    <div className="min-h-screen flex flex-col bg-white text-[#1d1d1f] antialiased">

      {/* Header */}
      <Header
        onOpenAuth={() => setShowAuthModal(true)}
        onOpenProfile={() => setShowProfileModal(true)}
        onOpenAddKebab={() => setShowAddModal(true)}
        onOpenFormula={() => setShowFormulaModal(true)}
      />

      {/* Hero Section — Minimal Apple / Samsung Style */}
      <section className="border-b border-gray-100 bg-[#fafafa]">
        <div className="max-w-6xl mx-auto px-4 sm:px-6 py-14 sm:py-20 text-center">
          <span className="inline-block text-xs font-semibold uppercase tracking-widest
text-red-600 mb-3">
            Guía Oficial • Algeciras
          </span>
          <h1 className="text-4xl sm:text-6xl font-bold tracking-tight text-[#1d1d1f] max-
w-3xl mx-auto leading-tight">
            Los mejores kebabs, clasificados con precisión.
          </h1>
          <p className="mt-4 text-base sm:text-lg text-[#86868b] max-w-xl mx-auto leading-
relaxed">
            Evaluaciones objetivas por criterios: calidad de carne, salsas, atención,
precio y local.
          </p>

          {/* Key Metrics */}
          <div className="mt-10 flex flex-wrap items-center justify-center gap-8 sm:gap-14
text-center">
            <div>
              <span className="block text-3xl sm:text-4xl font-bold text-[#1d1d1f]">
                {kebabs.length}
```

```
              </span>
              <span className="text-xs text-[#86868b] uppercase tracking-wider font-medium
mt-0.5 block">
                Locales Registrados
              </span>
            </div>

            <div className="w-px h-10 bg-gray-200 hidden sm:block" />

            <div>
              <span className="block text-3xl sm:text-4xl font-bold text-red-600">
                {stats.totalVotes}
              </span>
              <span className="text-xs text-[#86868b] uppercase tracking-wider font-medium
mt-0.5 block">
                Valoraciones
              </span>
            </div>

            {stats.topKebab && (
              <>
                <div className="w-px h-10 bg-gray-200 hidden sm:block" />
                <div>
                  <div className="flex items-center justify-center gap-1.5">
                    <Star className="w-5 h-5 text-red-600 fill-current" />
                    <span className="text-3xl sm:text-4xl font-bold text-[#1d1d1f]">
                      {stats.topKebab.average_score?.toFixed(1)}
                    </span>
                  </div>
                  <span className="text-xs text-[#86868b] uppercase tracking-wider font-
medium mt-0.5 block truncate max-w-[180px]">
                    N.º 1: {stats.topKebab.name}
                  </span>
                </div>
              </>
            )}
          </div>
        </div>
      </section>

      {/* Main Content */}
      <main className="flex-1 max-w-6xl w-full mx-auto px-4 sm:px-6 py-10 space-y-8">

        {/* Controls: Search, Sort & Zones */}
        <div className="space-y-4">
          <div className="flex flex-col sm:flex-row items-stretch sm:items-center justify-
between gap-3">
```

```
            <div className="relative flex-1">
              <Search className="absolute left-3.5 top-1/2 -translate-y-1/2 w-4 h-4 text-
gray-400" />
              <input
                type="text"
                value={searchTerm}
                onChange={(e) => setSearchTerm(e.target.value)}
                placeholder="Buscar por nombre o zona..."
                className="w-full pl-10 pr-4 py-2.5 text-sm bg-gray-50 border border-gray-
200 rounded-xl text-[#1d1d1f] placeholder-gray-400 focus:outline-none focus:border-red-500
focus:ring-1 focus:ring-red-500/20 transition-all"
              />
            </div>

            <div className="flex items-center gap-2">
              <div className="flex items-center gap-2 px-3.5 py-2 bg-gray-50 border border-
gray-200 rounded-xl text-xs font-medium text-gray-600">
                <Filter className="w-3.5 h-3.5 text-gray-400" />
                <span className="hidden sm:inline">Ordenar:</span>
                <select
                  value={sortBy}
                  onChange={(e) => setSortBy(e.target.value)}
                  className="bg-transparent text-[#1d1d1f] font-semibold focus:outline-none
cursor-pointer"
                >
                  <option value="rating">Puntuación</option>
                  <option value="votes">Más Valorados</option>
                  <option value="name">Nombre</option>
                </select>
              </div>
            </div>
          </div>

          {/* Zone Filter Chips */}
          <div className="flex items-center gap-2 overflow-x-auto pb-2 scrollbar-none">
            {zones.map(zone => {
              const count = zone === 'Todos' ? kebabs.length : kebabs.filter(k => k.zone
=== zone).length;
              const isSelected = selectedZone === zone;

              return (
                <button
                  key={zone}
                  onClick={() => setSelectedZone(zone)}
                  className={`px-3.5 py-1.5 rounded-full text-xs font-medium whitespace-
nowrap transition-colors flex items-center gap-2 shrink-0 ${
```

```
                    isSelected
                      ? 'bg-red-600 text-white'
                      : 'bg-gray-100 text-gray-600 hover:bg-gray-200'
                  }`}
                >
                  <span>{zone}</span>
                  <span className={`text-[10px] px-1.5 py-0.2 rounded-full ${
                    isSelected ? 'bg-white/20 text-white' : 'bg-gray-200 text-gray-500'
                  }`}>
                    {count}
                  </span>
                </button>
              );
            })}
          </div>
        </div>

        {/* Kebabs Grid */}
        {loading ? (
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
            {[1, 2, 3, 4, 5, 6].map(i => (
              <div key={i} className="h-80 rounded-2xl bg-gray-50 animate-pulse border
border-gray-100" />
            ))}
          </div>
        ) : filteredKebabs.length > 0 ? (
          <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
            {filteredKebabs.map((kebab, idx) => (
              <KebabCard
                key={kebab.id}
                kebab={kebab}
                rank={sortBy === 'rating' ? idx + 1 : null}
                onClick={() => setSelectedKebab(kebab)}
                onRateClick={(k) => {
                  if (!user) {
                    setShowAuthModal(true);
                  } else {
                    setRatingKebab(k);
                  }
                }}
                currentUserId={user?.id}
              />
            ))}
          </div>
        ) : (
```

```
          <div className="text-center py-20 px-4 rounded-2xl bg-gray-50 border border-gray-
100">
            <h3 className="text-base font-semibold text-[#1d1d1f]">
              No se han encontrado resultados
            </h3>
            <p className="text-xs text-[#86868b] mt-1 max-w-sm mx-auto">
              Prueba con otro término de búsqueda o selecciona otra zona.
            </p>
          </div>
        )}

      </main>

      {/* Minimal Footer */}
      <footer className="border-t border-gray-100 bg-white py-8 text-xs text-[#86868b]">
        <div className="max-w-6xl mx-auto px-4 sm:px-6 flex flex-col sm:flex-row items-
center justify-between gap-4">
          <div className="flex items-center gap-2">
            <span className="font-semibold text-[#1d1d1f]">AlgeKebab</span>
            <span>•</span>
            <span>Algeciras</span>
          </div>

          <p className="text-gray-400">
            © {new Date().getFullYear()} AlgeKebab. Todos los derechos reservados.
          </p>
        </div>
      </footer>

      {/* Modals */}
      {selectedKebab && (
        <KebabDetailModal
          kebab={selectedKebab}
          currentUserId={user?.id}
          onClose={() => setSelectedKebab(null)}
          onRateClick={(k) => {
            if (!user) {
              setShowAuthModal(true);
            } else {
              setRatingKebab(k);
            }
          }}
          onDeleteRating={handleDeleteRating}
        />
      )}

      {ratingKebab && (
```

```
        <RatingModal
          kebab={ratingKebab}
          existingRating={ratingKebab.ratings?.find(r => r.user_id === user?.id)}
          onClose={() => setRatingKebab(null)}
          onSave={handleSaveRating}
        />
      )}

      {showAuthModal && (
        <AuthModal
          onClose={() => setShowAuthModal(false)}
        />
      )}

      {showProfileModal && (
        <ProfileModal
          totalUserRatings={stats.myVotesCount}
          onClose={() => setShowProfileModal(false)}
        />
      )}

      {showAddModal && (
        <AddKebabModal
          onClose={() => setShowAddModal(false)}
          onAdd={handleAddKebab}
        />
      )}

      {showFormulaModal && (
        <FormulaInfoModal
          onClose={() => setShowFormulaModal(false)}
        />
      )}

    </div>
  );
}
```

4.5. Logotipo: `client/src/components/Logo.jsx`

```
jsx
const sizeMap = {
  sm: { heading: 'text-xl', subtitle: 'text-[10px]', tracking: 'tracking-tight' },
  md: { heading: 'text-3xl', subtitle: 'text-xs', tracking: 'tracking-tight' },
  lg: { heading: 'text-5xl', subtitle: 'text-sm', tracking: 'tracking-tight' },
};

export default function Logo({ size = 'md', showText = true }) {
  const s = sizeMap[size] || sizeMap.md;

  return (
    <div className="flex flex-col items-start select-none">
      <h1 className={`${s.heading} ${s.tracking} font-semibold leading-none`}>
        <span style={{ color: '#1d1d1f' }}>Alge</span>
        <span style={{ color: '#dc2626' }}>Kebab</span>
      </h1>
      {showText && (
        <span
          className={`${s.subtitle} font-medium uppercase tracking-widest mt-0.5`}
          style={{ color: '#86868b' }}
        >
          Algeciras
        </span>
      )}
    </div>
  );
}
```

4.6. Barra Superior: `client/src/components/Header.jsx`

```
jsx
import React from 'react';
import Logo from './Logo';
import { useAuth } from '../context/AuthContext';
import { LogIn, PlusCircle, Settings, User } from 'lucide-react';

const Header = ({ onOpenFormula, onOpenAddKebab, onOpenProfile, onOpenAuth }) => {
  const { user } = useAuth();

  return (
    <header className="sticky top-0 z-50 bg-white/95 backdrop-blur-sm border-b border-gray-
100">
      <div className="max-w-6xl mx-auto px-4 sm:px-6">
        <div className="flex items-center justify-between h-16">
          <Logo size="md" />

          <div className="flex items-center gap-2 sm:gap-3">
            <button
              onClick={onOpenFormula}
              className="flex items-center gap-1.5 px-3 py-2 text-sm font-medium text-
[#86868b] hover:text-[#1d1d1f] transition-colors rounded-lg hover:bg-gray-50"
            >
              <Settings size={16} />
              <span className="hidden sm:inline">Criterios</span>
            </button>

            {user && (
              <button
                onClick={onOpenAddKebab}
                className="flex items-center gap-1.5 px-4 py-2 text-sm font-semibold text-
white bg-[#dc2626] hover:bg-[#b91c1c] transition-colors rounded-lg"
              >
                <PlusCircle size={16} />
                <span className="hidden sm:inline">Nuevo Kebab</span>
              </button>
            )}

            {user ? (
              <button
                onClick={onOpenProfile}
                className="flex items-center justify-center w-9 h-9 rounded-full bg-
[#dc2626] text-white text-sm font-semibold hover:bg-[#b91c1c] transition-colors"
              >
```

```
                {user.username ? user.username.charAt(0).toUpperCase() : <User size={16}
/>}
              </button>
            ) : (
              <button
                onClick={onOpenAuth}
                className="flex items-center gap-1.5 px-4 py-2 text-sm font-semibold text-
[#dc2626] border border-[#dc2626]/30 hover:border-[#dc2626] hover:bg-red-50 transition-
colors rounded-lg"
              >
                <LogIn size={16} />
                <span>Entrar</span>
              </button>
            )}
          </div>
        </div>
      </div>
    </header>
  );
};

export default Header;
```

4.7. Tarjeta de Kebab: `client/src/components/KebabCard.jsx`

```
jsx
import React from 'react';
import { MapPin, Star, Navigation, ChevronRight } from 'lucide-react';
import { getScoreBadgeColor } from '../utils/formula';

export default function KebabCard({ kebab, rank, onClick, onRateClick, currentUserId }) {
  const userRating = kebab.ratings?.find(r => r.user_id === currentUserId);

  const mapsUrl =
    kebab.google_maps_url ||
    `https://www.google.com/maps/dir/?api=1&destination=${encodeURIComponent(kebab.name + '
' + kebab.address + ' Algeciras')}`;

  return (
    <div
      onClick={onClick}
      className="group relative bg-white border border-gray-100 rounded-2xl overflow-hidden
transition-all duration-300 cursor-pointer hover:shadow-lg flex flex-col justify-between"
    >
      <div>
        {/* Kebab Image */}
        <div className="h-48 w-full relative overflow-hidden">
          {kebab.image_url ? (
            <img
              src={kebab.image_url}
              alt={kebab.name}
              className="w-full h-full object-cover rounded-t-2xl group-hover:scale-[1.03]
transition-transform duration-500"
              loading="lazy"
            />
          ) : (
            <div className="w-full h-full bg-gray-100 rounded-t-2xl flex items-center
justify-center">
              <span className="text-gray-300 text-4xl">🌯</span>
            </div>
          )}

          {/* Zone badge */}
          <div className="absolute bottom-3 left-3 flex items-center gap-1.5 text-xs text-
gray-600 font-medium px-3 py-1.5 rounded-full bg-white/80 backdrop-blur-sm">
            <MapPin className="w-3.5 h-3.5 text-gray-500 shrink-0" />
            <span className="truncate max-w-[180px]">{kebab.zone}</span>
          </div>
```

```
          {/* Score badge */}
          {kebab.average_score !== null && kebab.average_score !== undefined ? (
            <div className="absolute top-3 right-3 flex items-center gap-1 px-3 py-1.5
rounded-full bg-red-600 text-white text-sm font-semibold shadow-sm">
              <Star className="w-3.5 h-3.5 fill-current" />
              <span>{kebab.average_score.toFixed(1)}</span>
            </div>
          ) : (
            <div className="absolute top-3 right-3 flex items-center gap-1 px-3 py-1.5
rounded-full bg-gray-200 text-gray-400 text-sm font-medium">
              <span>—</span>
            </div>
          )}
        </div>

        {/* Info Content */}
        <div className="p-5">
          <h3 className="text-lg font-semibold text-gray-900 line-clamp-1">
            {kebab.name}
          </h3>

          <p className="text-sm text-gray-500 line-clamp-1 mt-0.5">
            {kebab.address}
          </p>

          {kebab.description && (
            <p className="text-sm text-gray-600 line-clamp-2 mt-2 leading-relaxed">
              {kebab.description}
            </p>
          )}

          {/* Cómo llegar directo */}
          <a
            href={mapsUrl}
            target="_blank"
            rel="noopener noreferrer"
            onClick={(e) => e.stopPropagation()}
            className="inline-flex items-center gap-1.5 text-sm text-red-600 hover:text-
red-700 font-medium mt-3 transition-colors"
          >
            <Navigation className="w-3.5 h-3.5" />
            <span>Cómo llegar</span>
          </a>

          {/* Friend rating pills */}
          <div className="border-t border-gray-50 pt-4 mt-4">
            {kebab.ratings && kebab.ratings.length > 0 ? (
```

```
              <div className="flex flex-wrap gap-2">
                {kebab.ratings.map(r => (
                  <div
                    key={r.id || r.user_id}
                    className="flex items-center gap-1.5 pl-1.5 pr-2.5 py-1 rounded-full
bg-gray-100 text-xs"
                  >
                    <div
                      className="w-5 h-5 rounded-full flex items-center justify-center
text-[10px] font-semibold text-white uppercase"
                      style={{ backgroundColor: r.avatar_color || '#dc2626' }}
                    >
                      {r.username ? r.username.slice(0, 1) : '?'}
                    </div>
                    <span className="text-gray-700 font-medium">{r.username}</span>
                    <span className="font-semibold text-gray-900">
                      {r.final_score ? parseFloat(r.final_score).toFixed(1) : '-'}
                    </span>
                  </div>
                ))}
              </div>
            ) : (
              <p className="text-xs text-gray-400">
                Aún no hay valoraciones. ¡Sé el primero!
              </p>
            )}
          </div>
        </div>
      </div>

      {/* Footer */}
      <div className="px-5 pb-5 pt-2 flex items-center justify-between text-xs">
        <span className="flex items-center gap-1 text-gray-400 group-hover:text-gray-600
font-medium transition-colors">
          Ver detalles
          <ChevronRight className="w-3.5 h-3.5 group-hover:translate-x-0.5 transition-
transform" />
        </span>

        {userRating && (
          <span className="text-xs font-semibold text-red-600">
            Tu nota: {parseFloat(userRating.final_score).toFixed(1)}
          </span>
        )}
      </div>
    </div>
```

```
  );
}
```

4.8. Ficha Detallada: `client/src/components/KebabDetailModal.jsx`

```
jsx
import React, { useState } from 'react';
import { X, Star, MapPin, Navigation, Edit3, Trash2, ChevronDown, ChevronUp } from 'lucide-
react';
import { CRITERIA_CONFIG, calculateContribution, getScoreBadgeColor } from
'../utils/formula';

export default function KebabDetailModal({ kebab, onClose, onRateClick, currentUserId,
onDeleteRating }) {
  const [expandedUser, setExpandedUser] = useState(null);

  if (!kebab) return null;

  const userRating = kebab.ratings?.find(r => r.user_id === currentUserId);

  const mapsUrl =
    kebab.google_maps_url ||
    `https://www.google.com/maps/dir/?api=1&destination=${encodeURIComponent(kebab.name + '
' + kebab.address + ' Algeciras')}`;

  return (
    <div
      className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/40
backdrop-blur-sm"
      onClick={onClose}
    >
      <div
        className="relative w-full max-w-2xl max-h-[90vh] bg-white rounded-2xl shadow-xl
flex flex-col overflow-hidden"
        onClick={(e) => e.stopPropagation()}
      >

        {/* Header */}
        <div className="p-6 border-b border-gray-100 flex items-start justify-between gap-
4">
          <div className="space-y-2 min-w-0 flex-1">
            <span className="inline-flex items-center text-xs font-medium px-3 py-1
rounded-full bg-red-50 text-red-600">
              {kebab.zone}
            </span>

            <h2 className="text-2xl font-bold text-[#1d1d1f] tracking-tight">
              {kebab.name}
            </h2>
```

```
            <p className="text-sm text-[#86868b] flex items-center gap-1.5">
              <MapPin className="w-4 h-4 text-[#86868b] shrink-0" />
              <span>{kebab.address}</span>
            </p>
          </div>

          <div className="flex items-center gap-3 shrink-0">
            {kebab.average_score !== null && kebab.average_score !== undefined ? (
              <div className="flex flex-col items-center justify-center px-4 py-2.5
rounded-2xl bg-red-600 text-white">
                <div className="flex items-center gap-1.5 font-bold text-2xl leading-none">
                  <Star className="w-5 h-5 fill-current" />
                  <span>{kebab.average_score.toFixed(1)}</span>
                </div>
                <span className="text-[10px] uppercase font-medium tracking-wider text-red-
100 mt-1">
                  Media ({kebab.ratings?.length || 0})
                </span>
              </div>
            ) : (
              <div className="flex flex-col items-center justify-center px-4 py-2.5
rounded-2xl bg-gray-100 text-gray-400">
                <div className="flex items-center gap-1.5 font-bold text-2xl leading-none">
                  <Star className="w-5 h-5" />
                  <span>—</span>
                </div>
                <span className="text-[10px] uppercase font-medium tracking-wider text-
gray-400 mt-1">
                  Sin votos
                </span>
              </div>
            )}

            <button
              onClick={onClose}
              className="p-2 rounded-xl text-gray-400 hover:text-gray-600 hover:bg-gray-100
transition-colors"
            >
              <X className="w-5 h-5" />
            </button>
          </div>
        </div>

        {/* Content Body */}
        <div className="p-6 overflow-y-auto space-y-6">

          {/* Cómo llegar */}
```

```
          <div className="p-4 rounded-xl bg-gray-50 flex items-center justify-between gap-
4">
            <div className="flex items-center gap-3 min-w-0">
              <Navigation className="w-5 h-5 text-red-600 shrink-0" />
              <div className="min-w-0">
                <p className="text-sm font-medium text-gray-900">Ubicación y ruta</p>
                <p className="text-xs text-gray-500 truncate">{kebab.address}</p>
              </div>
            </div>

            <a
              href={mapsUrl}
              target="_blank"
              rel="noopener noreferrer"
              className="inline-flex items-center gap-1.5 px-4 py-2 text-sm font-semibold
text-white bg-red-600 hover:bg-red-700 rounded-xl transition-colors shrink-0"
            >
              <Navigation className="w-3.5 h-3.5" />
              <span>Abrir en Google Maps</span>
            </a>
          </div>

          {/* Action to Rate */}
          <div className="p-5 rounded-2xl bg-gray-50 flex flex-col sm:flex-row items-start
sm:items-center justify-between gap-4">
            <div>
              <h4 className="text-sm font-semibold text-gray-900">
                {userRating ? 'Tu valoración de este kebab' : '¿Has probado este kebab?'}
              </h4>
              <p className="text-xs text-gray-500 mt-0.5">
                {userRating
                  ? `Tu nota actual es ${parseFloat(userRating.final_score).toFixed(1)} /
10. Puedes editarla en cualquier momento.`
                  : 'Puntúa el kebab con la fórmula oficial (carne, salsas, atención,
precio y local).'}
              </p>
            </div>

            <button
              onClick={() => onRateClick(kebab)}
              className="flex items-center gap-2 px-4 py-2 rounded-xl text-xs font-semibold
text-white bg-red-600 hover:bg-red-700 transition-colors shrink-0"
            >
              <Edit3 className="w-4 h-4" />
              <span>{userRating ? 'Editar nota' : 'Puntuar'}</span>
            </button>
```

```
          </div>

          {/* Friends Ratings List */}
          <div>
            <div className="flex items-center justify-between mb-3">
              <h3 className="text-sm font-semibold uppercase tracking-wider text-gray-500">
                Valoraciones ({kebab.ratings?.length || 0})
              </h3>
            </div>

            {kebab.ratings && kebab.ratings.length > 0 ? (
              <div className="space-y-3">
                {kebab.ratings.map(rating => {
                  const isCurrent = rating.user_id === currentUserId;
                  const isExpanded = expandedUser === rating.user_id;

                  return (
                    <div
                      key={rating.id || rating.user_id}
                      className="p-4 rounded-xl border border-gray-100 hover:border-gray-
200 transition-colors"
                    >
                      <div className="flex items-center justify-between">
                        <div className="flex items-center gap-3">
                          <div
                            className="w-9 h-9 rounded-full flex items-center justify-
center font-semibold text-sm text-white uppercase"
                            style={{ backgroundColor: rating.avatar_color || '#dc2626' }}
                          >
                            {rating.username ? rating.username.slice(0, 1) : '?'}
                          </div>
                          <div>
                            <div className="flex items-center gap-2">
                              <span className="font-semibold text-gray-900 text-sm">
                                {rating.username}
                              </span>
                              {isCurrent && (
                                <span className="text-[10px] font-semibold text-red-600 bg-
red-50 px-2 py-0.5 rounded-full">
                                  Tú
                                </span>
                              )}
                            </div>
                            <span className="text-xs text-gray-400">
                              {new Date(rating.updated_at ||
rating.created_at).toLocaleDateString('es-ES', {
```

```
                                day: 'numeric',
                                month: 'short',
                                year: 'numeric'
                              })}
                            </span>
                          </div>
                        </div>

                        <div className="flex items-center gap-3">
                          <div className="flex items-center gap-1 text-sm font-bold text-
gray-900">
                            <Star className="w-4 h-4 text-red-600 fill-current" />
                            <span>{parseFloat(rating.final_score).toFixed(1)}</span>
                          </div>

                          <button
                            onClick={() => setExpandedUser(isExpanded ? null :
rating.user_id)}
                            className="p-1.5 rounded-lg text-gray-400 hover:text-gray-600
hover:bg-gray-50 transition-colors"
                          >
                            {isExpanded ? <ChevronUp className="w-4 h-4" /> : <ChevronDown
className="w-4 h-4" />}
                          </button>

                          {isCurrent && (
                            <button
                              onClick={() => {
                                if (window.confirm('¿Seguro que quieres borrar tu
valoración?')) {
                                  onDeleteRating(kebab.id);
                                }
                              }}
                              className="p-1.5 rounded-lg text-red-500 hover:text-red-700
hover:bg-red-50 transition-colors"
                            >
                              <Trash2 className="w-4 h-4" />
                            </button>
                          )}
                        </div>
                      </div>

                      {rating.comment && (
                        <p className="mt-3 text-xs text-gray-600 bg-gray-50 p-3 rounded-lg
leading-relaxed">
                          {rating.comment}
                        </p>
```

```
                    )}

                    {/* Criteria breakdown */}
                    {isExpanded && (
                      <div className="mt-3 pt-3 border-t border-gray-100 grid grid-cols-2
sm:grid-cols-5 gap-2 text-center">
                        {Object.entries(CRITERIA_CONFIG).map(([key, config]) => {
                          const rawScore = rating[`${key}_score`];
                          const contrib = calculateContribution(rawScore,
config.maxPoints);

                          return (
                            <div key={key} className="p-2 rounded-lg bg-gray-50">
                              <span className="text-[10px] text-gray-400 uppercase font-
medium block truncate">
                                {config.label}
                              </span>
                              <span className="text-xs font-semibold text-gray-900 block
mt-0.5">
                                {parseFloat(rawScore || 0).toFixed(1)} / 10
                              </span>
                              <span className="text-[10px] font-medium text-red-600 block
mt-0.5">
                                +{contrib.toFixed(2)} pts
                              </span>
                            </div>
                          );
                        })}
                      </div>
                    )}
                  </div>
                );
              })}
            </div>
          ) : (
            <p className="text-xs text-gray-400 italic">
              Aún no hay valoraciones para este local.
            </p>
          )}
        </div>

      </div>
    </div>
  );
}
```

4.9. Formulario de Votación: `client/src/components/RatingModal.jsx`

```
jsx
import React, { useState, useEffect } from 'react';
import { X, Star, Save, AlertCircle } from 'lucide-react';
import { CRITERIA_CONFIG, calculateFormulaScore, calculateContribution } from
'../utils/formula';

export default function RatingModal({ kebab, existingRating, onClose, onSave }) {
  const [scores, setScores] = useState({
    kebab_score: 7.0,
    ubicacion_score: 8.0,
    atencion_score: 8.0,
    calidad_precio_score: 7.0,
    local_presentacion_score: 7.0
  });

  const [comment, setComment] = useState('');
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState('');

  useEffect(() => {
    if (existingRating) {
      setScores({
        kebab_score: parseFloat(existingRating.kebab_score) || 7.0,
        ubicacion_score: parseFloat(existingRating.ubicacion_score) || 8.0,
        atencion_score: parseFloat(existingRating.atencion_score) || 8.0,
        calidad_precio_score: parseFloat(existingRating.calidad_precio_score) || 7.0,
        local_presentacion_score: parseFloat(existingRating.local_presentacion_score) ||
7.0
      });
      setComment(existingRating.comment || '');
    }
  }, [existingRating]);

  const liveFinalScore = calculateFormulaScore(scores);

  const handleSliderChange = (criterionId, val) => {
    setScores(prev => ({
      ...prev,
      [criterionId]: parseFloat(val)
    }));
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
```

```
    setError('');
    setSubmitting(true);

    try {
      await onSave({
        kebab_id: kebab.id,
        ...scores,
        comment
      });
      onClose();
    } catch (err) {
      setError(err.message || 'Error al guardar la puntuación');
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <div
      className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/40
backdrop-blur-sm"
      onClick={onClose}
    >
      <div
        className="relative w-full max-w-lg bg-white rounded-2xl shadow-xl overflow-hidden
flex flex-col max-h-[90vh]"
        onClick={(e) => e.stopPropagation()}
      >

        {/* Header */}
        <div className="p-6 border-b border-gray-100 flex items-center justify-between">
          <div>
            <span className="text-xs font-semibold uppercase tracking-wider text-red-600">
              Valorar
            </span>
            <h3 className="text-xl font-bold text-[#1d1d1f] line-clamp-1 mt-0.5">
              {kebab.name}
            </h3>
          </div>

          <button
            onClick={onClose}
            className="p-2 rounded-xl text-gray-400 hover:text-gray-600 hover:bg-gray-100
transition-colors"
          >
            <X className="w-5 h-5" />
```

```
          </button>
        </div>

        {/* Form Body */}
        <form onSubmit={handleSubmit} className="p-6 space-y-6 overflow-y-auto flex-1">

          {/* Live Score Preview */}
          <div className="p-4 rounded-xl bg-gray-50 border border-gray-200 flex items-
center justify-between">
            <div>
              <span className="text-xs font-medium text-gray-600">Nota final
calculada</span>
              <p className="text-[11px] text-gray-400 mt-0.5">
                Kebab (6 pts) + Ubicación (1) + Atención (1) + Precio (1) + Local (1)
              </p>
            </div>

            <div className="px-4 py-2 rounded-xl bg-red-600 text-white flex items-center
gap-1.5 text-2xl font-bold shadow-sm">
              <Star className="w-5 h-5 fill-current" />
              <span>{liveFinalScore.toFixed(1)}</span>
            </div>
          </div>

          {error && (
            <div className="p-3.5 rounded-xl bg-red-50 border border-red-100 text-xs text-
red-600 flex items-center gap-2">
              <AlertCircle className="w-4 h-4 shrink-0" />
              <span>{error}</span>
            </div>
          )}

          {/* Criteria Sliders */}
          <div className="space-y-4">
            {CRITERIA_CONFIG.map(crit => {
              const currentVal = scores[crit.id];
              const contrib = calculateContribution(currentVal, crit.maxPoints);

              return (
                <div key={crit.id} className="p-4 rounded-xl bg-gray-50/70 border border-
gray-200/80 space-y-2.5">
                  <div className="flex items-center justify-between">
                    <div>
                      <span className="text-sm font-semibold text-[#1d1d1f]">{crit.label}
</span>
                      <span className="text-xs text-gray-400 ml-2">
                        (hasta {crit.maxPoints} pts)
```

```
                    </span>
                  </div>

                  <div className="text-sm font-bold text-red-600 px-2 py-0.5 rounded bg-
white border border-gray-200 shadow-sm">
                    {currentVal.toFixed(1)} / 10
                  </div>
                </div>

                <div className="flex items-center justify-between text-[11px] text-gray-
500">
                  <span>{crit.description}</span>
                  <span className="font-semibold text-gray-700">
                    Suma: +{contrib.toFixed(2)} pts
                  </span>
                </div>

                <input
                  type="range"
                  min="0"
                  max="10"
                  step="0.1"
                  value={currentVal}
                  onChange={(e) => handleSliderChange(crit.id, e.target.value)}
                  className="w-full cursor-pointer"
                />

                <div className="flex justify-between text-[10px] text-gray-400">
                  <span>0</span>
                  <span>5</span>
                  <span>10</span>
                </div>
              </div>
            );
          })}
        </div>

        {/* Comment */}
        <div>
          <label className="block text-xs font-medium uppercase tracking-wider text-gray-
500 mb-1.5">
            Comentario u opinión personal (opcional)
          </label>
          <textarea
            rows="3"
            value={comment}
            onChange={(e) => setComment(e.target.value)}
```

```
              placeholder="¿Qué tal el pan, la carne, las salsas o el trato?"
              className="w-full px-4 py-2.5 text-sm bg-gray-50 border border-gray-200
rounded-xl text-[#1d1d1f] placeholder-gray-400 focus:outline-none focus:border-red-500
focus:ring-1 focus:ring-red-500/20 transition-all resize-none"
            />
          </div>

          {/* Footer Submit */}
          <div className="pt-3 flex items-center justify-end gap-3 border-t border-gray-
100">
            <button
              type="button"
              onClick={onClose}
              className="px-4 py-2 rounded-xl text-sm font-medium text-gray-600 hover:text-
gray-900 hover:bg-gray-100 transition-colors"
            >
              Cancelar
            </button>

            <button
              type="submit"
              disabled={submitting}
              className="flex items-center gap-2 px-6 py-2.5 rounded-xl text-sm font-
semibold text-white bg-red-600 hover:bg-red-700 shadow-sm disabled:opacity-50 transition-
colors"
            >
              <Save className="w-4 h-4" />
              <span>{submitting ? 'Guardando...' : (existingRating ? 'Actualizar Nota' :
'Guardar Puntuación')}</span>
            </button>
          </div>

        </form>

      </div>
    </div>
  );
}
```

4.10. Modal de Autenticación: `client/src/components/AuthModal.jsx`

```
jsx
import React, { useState } from 'react';
import Logo from './Logo';
import { useAuth } from '../context/AuthContext';
import { X, LogIn, UserPlus, Lock, User, AlertCircle } from 'lucide-react';

const AuthModal = ({ onClose }) => {
  const [isLogin, setIsLogin] = useState(true);
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const { login, register } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);

    try {
      if (isLogin) {
        await login(username, password);
      } else {
        await register(username, password);
      }
      onClose();
    } catch (err) {
      setError(err.message || 'Error al autenticar');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div
      className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/40
backdrop-blur-sm"
      onClick={onClose}
    >
      <div
        className="relative w-full max-w-md bg-white rounded-2xl shadow-xl overflow-hidden"
        onClick={(e) => e.stopPropagation()}
      >
```

```
        {/* Header */}
        <div className="flex items-center justify-between p-6 border-b border-gray-100">
          <Logo size="sm" />
          <button
            onClick={onClose}
            className="p-1 rounded-lg text-gray-400 hover:text-gray-600 hover:bg-gray-100
transition-colors"
          >
            <X size={20} />
          </button>
        </div>

        {/* Tab switcher */}
        <div className="flex border-b border-gray-100">
          <button
            type="button"
            onClick={() => { setIsLogin(true); setError(''); }}
            className={`flex-1 py-3 text-sm font-semibold transition-colors border-b-2 -mb-
px ${
              isLogin
                ? 'border-[#dc2626] text-[#dc2626]'
                : 'border-transparent text-gray-400 hover:text-gray-600'
            }`}
          >
            Iniciar Sesión
          </button>
          <button
            type="button"
            onClick={() => { setIsLogin(false); setError(''); }}
            className={`flex-1 py-3 text-sm font-semibold transition-colors border-b-2 -mb-
px ${
              !isLogin
                ? 'border-[#dc2626] text-[#dc2626]'
                : 'border-transparent text-gray-400 hover:text-gray-600'
            }`}
          >
            Crear Cuenta
          </button>
        </div>

        {/* Form */}
        <form onSubmit={handleSubmit} className="p-6 space-y-4">
          {error && (
            <div className="flex items-center gap-2 p-3 bg-red-50 text-[#dc2626] text-xs
font-medium rounded-xl">
              <AlertCircle size={16} className="shrink-0" />
```

```
              <span>{error}</span>
            </div>
          )}

          <div>
            <label className="block text-xs font-semibold uppercase tracking-wider text-
gray-500 mb-1.5">
              Usuario
            </label>
            <div className="relative">
              <User size={16} className="absolute left-3.5 top-1/2 -translate-y-1/2 text-
gray-400" />
              <input
                type="text"
                value={username}
                onChange={(e) => setUsername(e.target.value)}
                placeholder="Nombre de usuario"
                required
                className="w-full pl-10 pr-4 py-2.5 text-sm bg-gray-100 border border-gray-
200 rounded-xl text-[#1d1d1f] placeholder-gray-400 focus:outline-none focus:border-
[#dc2626] focus:ring-2 focus:ring-[#dc2626]/20 transition-all"
              />
            </div>
          </div>

          <div>
            <label className="block text-xs font-semibold uppercase tracking-wider text-
gray-500 mb-1.5">
              Contraseña
            </label>
            <div className="relative">
              <Lock size={16} className="absolute left-3.5 top-1/2 -translate-y-1/2 text-
gray-400" />
              <input
                type="password"
                value={password}
                onChange={(e) => setPassword(e.target.value)}
                placeholder="Contraseña"
                required
                className="w-full pl-10 pr-4 py-2.5 text-sm bg-gray-100 border border-gray-
200 rounded-xl text-[#1d1d1f] placeholder-gray-400 focus:outline-none focus:border-
[#dc2626] focus:ring-2 focus:ring-[#dc2626]/20 transition-all"
              />
            </div>
            <p className="mt-1.5 text-[11px] text-[#86868b]">
              Tu sesión se mantendrá iniciada en este dispositivo.
```

```
            </p>
          </div>

          <button
            type="submit"
            disabled={loading}
            className="w-full py-2.5 px-4 text-sm font-semibold text-white bg-[#dc2626]
hover:bg-[#b91c1c] disabled:opacity-50 transition-colors rounded-xl flex items-center
justify-center gap-2"
          >
            {loading ? (
              <span className="inline-block w-4 h-4 border-2 border-white/30 border-t-white
rounded-full animate-spin" />
            ) : isLogin ? (
              <>
                <LogIn size={16} />
                <span>Entrar</span>
              </>
            ) : (
              <>
                <UserPlus size={16} />
                <span>Registrarse</span>
              </>
            )}
          </button>
        </form>
      </div>
    </div>
  );
};

export default AuthModal;
```

4.11. Modal de Perfil: `client/src/components/ProfileModal.jsx`

```jsx
import React, { useState } from 'react';
import { X, User, Lock, LogOut, CheckCircle2, AlertCircle, Save } from 'lucide-react';
import { useAuth } from '../context/AuthContext';

export default function ProfileModal({ onClose, totalUserRatings }) {
  const { user, updateProfile, logout } = useAuth();

  const [newUsername, setNewUsername] = useState(user?.username || '');
  const [newPassword, setNewPassword] = useState('');
  const [currentPassword, setCurrentPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState('');
  const [error, setError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setMessage('');
    setLoading(true);

    try {
      const res = await updateProfile({
        new_username: newUsername !== user.username ? newUsername : undefined,
        new_password: newPassword ? newPassword : undefined,
        current_password: currentPassword || undefined
      });

      setMessage(res.message || 'Perfil actualizado con éxito');
      setNewPassword('');
      setCurrentPassword('');
    } catch (err) {
      setError(err.message || 'Error al actualizar el perfil');
    } finally {
      setLoading(false);
    }
  };

  const handleLogout = () => {
    logout();
    onClose();
  };

  return (
```

```
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/40
backdrop-blur-sm animate-fade-in">
      <div
        className="relative w-full max-w-md bg-white rounded-2xl shadow-xl overflow-hidden"
        onClick={(e) => e.stopPropagation()}
      >

        {/* Header */}
        <div className="p-6 border-b border-gray-100 flex items-center justify-between">
          <div className="flex items-center gap-3">
            <div
              className="w-10 h-10 rounded-full flex items-center justify-center font-
semibold text-base text-white uppercase"
              style={{ backgroundColor: user?.avatar_color || '#dc2626' }}
            >
              {user?.username ? user.username.slice(0, 1) : '?'}
            </div>
            <div>
              <h3 className="text-base font-semibold text-gray-900">
                {user?.username}
              </h3>
              <p className="text-xs text-gray-500">
                {totalUserRatings} {totalUserRatings === 1 ? 'kebab valorado' : 'kebabs
valorados'}
              </p>
            </div>
          </div>

          <button
            onClick={onClose}
            className="p-2 rounded-xl text-gray-400 hover:text-gray-600 hover:bg-gray-50
transition-colors"
          >
            <X className="w-5 h-5" />
          </button>
        </div>

        {/* Form */}
        <form onSubmit={handleSubmit} className="p-6 space-y-5">

          {message && (
            <div className="p-3 rounded-xl bg-green-50 text-sm text-green-700 flex items-
center gap-2">
              <CheckCircle2 className="w-4 h-4 shrink-0" />
              <span>{message}</span>
            </div>
```

```
          )}

          {error && (
            <div className="p-3 rounded-xl bg-red-50 text-sm text-red-600 flex items-center
gap-2">
              <AlertCircle className="w-4 h-4 shrink-0" />
              <span>{error}</span>
            </div>
          )}

          <div>
            <label className="block text-xs font-medium uppercase tracking-wider text-gray-
500 mb-1.5">
              Nombre de usuario
            </label>
            <div className="relative">
              <User className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-gray-
400" />
              <input
                type="text"
                value={newUsername}
                onChange={(e) => setNewUsername(e.target.value)}
                className="w-full pl-10 pr-4 py-2.5 text-sm bg-gray-50 border border-gray-
200 rounded-xl text-gray-900 placeholder-gray-400 focus:outline-none focus:border-red-500
focus:ring-1 focus:ring-red-500/20 transition-all"
              />
            </div>
          </div>

          <div className="pt-2 border-t border-gray-100">
            <label className="block text-xs font-medium uppercase tracking-wider text-gray-
500 mb-1.5">
              Cambiar contraseña
            </label>
            <div className="space-y-2">
              <div className="relative">
                <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-
gray-400" />
                <input
                  type="password"
                  value={newPassword}
                  onChange={(e) => setNewPassword(e.target.value)}
                  placeholder="Nueva contraseña (opcional)"
                  className="w-full pl-10 pr-4 py-2.5 text-sm bg-gray-50 border border-
gray-200 rounded-xl text-gray-900 placeholder-gray-400 focus:outline-none focus:border-red-
500 focus:ring-1 focus:ring-red-500/20 transition-all"
```

```
                />
              </div>

              {newPassword && (
                <div className="relative">
                  <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-
gray-400" />
                  <input
                    type="password"
                    value={currentPassword}
                    onChange={(e) => setCurrentPassword(e.target.value)}
                    placeholder="Contraseña actual para confirmar"
                    className="w-full pl-10 pr-4 py-2.5 text-sm bg-gray-50 border border-
gray-200 rounded-xl text-gray-900 placeholder-gray-400 focus:outline-none focus:border-red-
500 focus:ring-1 focus:ring-red-500/20 transition-all"
                  />
                </div>
              )}
            </div>
          </div>

          <div className="pt-3 flex items-center justify-between">
            <button
              type="button"
              onClick={handleLogout}
              className="flex items-center gap-1.5 px-3 py-2 rounded-xl text-sm font-medium
text-red-600 hover:bg-red-50 transition-colors"
            >
              <LogOut className="w-4 h-4" />
              <span>Cerrar sesión</span>
            </button>

            <button
              type="submit"
              disabled={loading}
              className="flex items-center gap-2 px-5 py-2.5 rounded-xl text-sm font-
semibold text-white bg-red-600 hover:bg-red-700 disabled:opacity-50 transition-colors"
            >
              <Save className="w-4 h-4" />
              <span>{loading ? 'Guardando...' : 'Guardar'}</span>
            </button>
          </div>

        </form>

      </div>
    </div>
```

```
  );
}
```

4.12. Modal Añadir Kebab: `client/src/components/AddKebabModal.jsx`

```
jsx
import React, { useState } from 'react';
import { X, Plus, MapPin, Store, AlertCircle } from 'lucide-react';

const COMMON_ZONES = [
  'Zona Puerto / Isla Verde',
  'Las Palomas',
  'Centro / Plaza Alta',
  'San García / Europa',
  'Getares',
  'Bajadilla / Reconquista',
  'Puerta Europa'
];

export default function AddKebabModal({ onClose, onAdd }) {
  const [name, setName] = useState('');
  const [zone, setZone] = useState('Zona Puerto / Isla Verde');
  const [address, setAddress] = useState('');
  const [description, setDescription] = useState('');
  const [googleMapsUrl, setGoogleMapsUrl] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!name.trim()) {
      setError('Introduce el nombre del kebab');
      return;
    }

    setError('');
    setLoading(true);

    try {
      await onAdd({
        name: name.trim(),
        zone,
        address: address.trim() || `Algeciras (${zone})`,
        description: description.trim(),
        google_maps_url: googleMapsUrl.trim()
      });
      onClose();
    } catch (err) {
      setError(err.message || 'Error al añadir el kebab');
```

```
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/40
backdrop-blur-sm">
      <div
        className="relative w-full max-w-md bg-white rounded-2xl shadow-xl overflow-hidden"
        onClick={(e) => e.stopPropagation()}
      >

        {/* Header */}
        <div className="p-5 border-b border-gray-100 flex items-center justify-between bg-
white">
          <div>
            <h3 className="text-base font-semibold text-gray-900">
              Añadir Nuevo Kebab en Algeciras
            </h3>
            <p className="text-[11px] text-gray-500 mt-0.5">
              Registra un local para que el grupo pueda empezar a puntuarlo
            </p>
          </div>

          <button
            onClick={onClose}
            className="p-1.5 rounded-lg text-gray-400 hover:text-gray-600 hover:bg-gray-100
transition-colors"
          >
            <X className="w-5 h-5" />
          </button>
        </div>

        {/* Form Body */}
        <form onSubmit={handleSubmit} className="p-5 space-y-4">

          {error && (
            <div className="p-3 rounded-xl bg-red-50 border border-red-100 text-xs text-
red-600 flex items-center gap-2">
              <AlertCircle className="w-4 h-4 shrink-0" />
              <span>{error}</span>
            </div>
          )}

          <div>
```

```
            <label className="block text-xs font-medium uppercase tracking-wider text-gray-
500 mb-1.5">
              Nombre del Kebab *
            </label>
            <div className="relative">
              <Store className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-gray-
400" />
              <input
                type="text"
                required
                value={name}
                onChange={(e) => setName(e.target.value)}
                placeholder="Ej: Kebab La Isla, Dchiqui..."
                className="w-full pl-9 pr-3 py-2 text-sm bg-gray-50 border border-gray-200
rounded-xl text-gray-900 placeholder-gray-400 focus:outline-none focus:border-red-500
transition-colors"
              />
            </div>
          </div>

          <div>
            <label className="block text-xs font-medium uppercase tracking-wider text-gray-
500 mb-1.5">
              Zona de Algeciras *
            </label>
            <select
              value={zone}
              onChange={(e) => setZone(e.target.value)}
              className="w-full px-3 py-2 text-sm bg-gray-50 border border-gray-200
rounded-xl text-gray-900 focus:outline-none focus:border-red-500 transition-colors"
            >
              {COMMON_ZONES.map(z => (
                <option key={z} value={z}>{z}</option>
              ))}
            </select>
          </div>

          <div>
            <label className="block text-xs font-medium uppercase tracking-wider text-gray-
500 mb-1.5">
              Dirección o Calle
            </label>
            <div className="relative">
              <MapPin className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-
gray-400" />
              <input
```

```
                type="text"
                value={address}
                onChange={(e) => setAddress(e.target.value)}
                placeholder="Ej: Calle Sindicalista Luis Cobos, Av. Virgen del Carmen..."
                className="w-full pl-9 pr-3 py-2 text-sm bg-gray-50 border border-gray-200
rounded-xl text-gray-900 placeholder-gray-400 focus:outline-none focus:border-red-500
transition-colors"
              />
            </div>
          </div>

          <div>
            <label className="block text-xs font-medium uppercase tracking-wider text-gray-
500 mb-1.5">
              Descripción o especialidad (opcional)
            </label>
            <textarea
              rows="2"
              value={description}
              onChange={(e) => setDescription(e.target.value)}
              placeholder="Ej: Famoso por sus salsas caseras, abre hasta tarde..."
              className="w-full px-3 py-2 text-sm bg-gray-50 border border-gray-200
rounded-xl text-gray-900 placeholder-gray-400 focus:outline-none focus:border-red-500
transition-colors resize-none"
            />
          </div>

          <div>
            <label className="block text-xs font-medium uppercase tracking-wider text-gray-
500 mb-1.5">
              Enlace de Google Maps (opcional)
            </label>
            <input
              type="url"
              value={googleMapsUrl}
              onChange={(e) => setGoogleMapsUrl(e.target.value)}
              placeholder="https://maps.google.com/?q=..."
              className="w-full px-3 py-2 text-sm bg-gray-50 border border-gray-200
rounded-xl text-gray-900 placeholder-gray-400 focus:outline-none focus:border-red-500
transition-colors"
            />
          </div>

          <div className="pt-2 flex items-center justify-end gap-3 border-t border-gray-
100">
            <button
```

```
              type="button"
              onClick={onClose}
              className="px-4 py-2 rounded-xl text-xs font-medium text-gray-500 hover:text-
gray-700 hover:bg-gray-100 transition-colors"
            >
              Cancelar
            </button>

            <button
              type="submit"
              disabled={loading}
              className="flex items-center gap-2 px-5 py-2.5 rounded-xl text-xs font-
semibold text-white bg-red-600 hover:bg-red-700 shadow-sm disabled:opacity-50 transition-
colors"
            >
              <Plus className="w-4 h-4" />
              <span>{loading ? 'Añadiendo...' : 'Añadir Kebab'}</span>
            </button>
          </div>

        </form>

      </div>
    </div>
  );
}
```

4.13. Modal de Criterios: `client/src/components/FormulaInfoModal.jsx`

```
jsx
import React from 'react';
import { X, Calculator } from 'lucide-react';
import { CRITERIA_CONFIG } from '../utils/formula';

export default function FormulaInfoModal({ onClose }) {
  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/40
backdrop-blur-sm">
      <div
        className="relative w-full max-w-lg bg-white rounded-2xl shadow-xl overflow-hidden"
        onClick={(e) => e.stopPropagation()}
      >

        {/* Header */}
        <div className="p-5 border-b border-gray-100 flex items-center justify-between bg-
white">
          <div className="flex items-center gap-3">
            <div className="p-2 rounded-xl bg-red-50 text-red-600 border border-red-100">
              <Calculator className="w-5 h-5" />
            </div>
            <div>
              <h3 className="text-base font-semibold text-gray-900">
                Fórmula Oficial de Puntuación
              </h3>
              <p className="text-[11px] text-gray-500 mt-0.5">
                Puntuaciones de 0 a 10 con conversión matemática
              </p>
            </div>
          </div>

          <button
            onClick={onClose}
            className="p-1.5 rounded-lg text-gray-400 hover:text-gray-600 hover:bg-gray-100
transition-colors"
          >
            <X className="w-5 h-5" />
          </button>
        </div>

        {/* Body */}
        <div className="p-5 space-y-4 max-h-[80vh] overflow-y-auto">
```

```
          <div className="p-3.5 rounded-xl bg-red-50 border border-red-100 text-xs text-
red-700 leading-relaxed">
            <strong>Cálculo Automático:</strong> Vosotros solo puntuáis del 0 al 10 con
comodidad. El sistema pondera cada nota automáticamente dando prioridad al sabor y la carne
(60%).
          </div>

          {/* Table of criteria */}
          <div className="space-y-2">
            {CRITERIA_CONFIG.map(crit => (
              <div
                key={crit.id}
                className="p-3 rounded-xl bg-gray-50 border border-gray-200 flex items-
center justify-between"
              >
                <div className="flex items-center gap-3">
                  <span className="text-xl">{crit.icon}</span>
                  <div>
                    <h4 className="text-xs font-semibold text-gray-900">{crit.label}</h4>
                    <p className="text-[11px] text-gray-500">{crit.description}</p>
                  </div>
                </div>

                <div className="text-right">
                  <span className="block text-xs font-bold text-red-600">
                    {crit.weightLabel}
                  </span>
                  <span className="text-[10px] text-gray-400">
                    {crit.percentage} del total
                  </span>
                </div>
              </div>
            ))}
          </div>

          <div className="p-3 rounded-xl bg-gray-50 border border-gray-200 text-xs text-
gray-600">
            <strong>Suma total:</strong> 6.0 + 1.0 + 1.0 + 1.0 + 1.0 = <strong>10.0 Puntos
Máximos</strong>.
          </div>

        </div>

      </div>
    </div>
  );
}
```

4.14. Punto de Entrada React: `client/src/main.jsx`

Monta el árbol React, inyecta el `AuthProvider` como contexto envolvente y enlaza los estilos globales de Tailwind en el DOM.

```
jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import { AuthProvider } from './context/AuthContext.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <AuthProvider>
      <App />
    </AuthProvider>
  </React.StrictMode>
);
```

4.15. Plantilla HTML Principal: `client/index.html`

Documento HTML5 base. Define las fuentes tipográficas del sistema (Inter), viewport responsive sin zoom accidental en móviles, favicon de kebab e inicializa el contenedor `#root`.

```
html
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="data:image/svg+xml,<svg
xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><text y='.9em' font-size='90'>
</text></svg>" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-
scale=1.0, user-scalable=no" />
    <title>AlgeKebab | La Ruta del Kebab en Algeciras</title>
    <!-- Google Fonts Inter -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700&display=swap" rel="stylesheet">
  </head>
  <body class="bg-white text-neutral-900 min-h-screen selection:bg-red-600 selection:text-
white font-sans antialiased">
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

4.16. Configuración de Vite: `client/vite.config.js`

Configuración del bundler Vite con plugin oficial de React y proxy de desarrollo apuntando las llamadas `/api` al backend Express (`http://localhost:3001`).

```
javascript
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:3001',
        changeOrigin: true
      }
    }
  }
});
```

5. Configuración de Paquetes y Dependencias

5.1. `package.json` (Raíz)

Define las dependencias de producción del backend (Express, PostgreSQL client, Bcrypt, JWT) y los scripts de compilación y arranque unificados.

```
json
{
  "name": "algekebab",
  "version": "1.0.0",
  "description": "App para puntuar kebabs en Algeciras con amigos",
  "main": "server/index.js",
  "scripts": {
    "start": "node server/index.js",
    "server": "node server/index.js",
    "client": "npm --prefix client run dev",
    "dev": "npm --prefix client run dev",
    "build": "npm --prefix client install && npm --prefix client run build"
  },
  "dependencies": {
    "@supabase/supabase-js": "^2.118.0",
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.2",
    "pg": "^8.13.1"
  }
}
```

5.2. `client/package.json` (Frontend)

Define las dependencias de interfaz: React 18, Lucide React (iconografía moderna y ligera), Tailwind CSS, Autoprefixer, PostCSS y el compilador Vite.

```
json
{
  "name": "client",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "lucide-react": "^0.475.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@types/react": "^18.3.18",
    "@types/react-dom": "^18.3.5",
    "@vitejs/plugin-react": "^4.3.4",
    "autoprefixer": "^10.4.20",
    "postcss": "^8.5.1",
    "tailwindcss": "^3.4.17",
    "vite": "^6.1.0"
  }
}
```

6. Variables de Entorno y Conexión en Alwaysdata

Para que la aplicación funcione en local o en Alwaysdata, solo necesita estas variables:

```
env
PORT=3001
JWT_SECRET=kebab-algeciras-secreto-2026
DATABASE_URL=postgresql://la-ruta-del-kebab:Villamil2412@postgresql-la-ruta-del-
kebab.alwaysdata.net:5432/la-ruta-del-kebab_kebab
```